Forecasting Flu Outbreaks with Time-Series LLM's: Leveraging Weather and Environmental Signals

A CAPSTONE PROJECT REPORT

Submitted in partial fulfillment of the requirement for the award of the Degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (AI &ML)

by

Chinnola Akash Rao (22BCE8371)
Aditya Nair (22BCE8612)
Nandana S Krishna (22BCE8963)
Edhara Jnaneswar Chowdary (22BCE9296)

Under the Guidance of

DR. S. KALYANI



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)
VIT-AP UNIVERSITY
AMARAVATI- 522237

December 2025

CERTIFICATE

This is to certify that the Capstone Project work titled **"Forecasting Flu Outbreaks with Time-Series LLMs: Leveraging Weather and Environmental Signals"**, submitted by **Chinnola Akash Rao (22BCE8371)**, **Aditya Nair (22BCE8612), Nandana S Krishna (22BCE8963), Edhara Jnaneswar Chowdary (22BCE9296)** is carried out in partial fulfillment of the requirements for the award of the **Bachelor of Technology in Computer Science and Engineering (AI & ML)** at **VIT-AP University. This** is a Bonafide record of original work done under my supervision and guidance. The contents of this report, in full or in part, have not been submitted to any other Institute or University for the award of any degree or diploma.



Dr. S. KALYANI
Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner **External Examiner**

Approved by

PROGRAM CHAIR **DEAN**

B. Tech. CSE (AIML) School Of Computer Science and Engineering

ACKNOWLEDGEMENTS

We would like to thank **Dr. S. Kalyani**, our project guide, for giving us her guidance and suport all through the capstone project. She was always encouraging us and giving feedbacks which really helped a lot. Without her proper guidance, we dont think the project would have come out this well.

We also want to thank the **School of Computer Science and Engineering (SCOPE), VIT-AP University** for providing the resources and infrastucture we needed. The faculty's and lab staff's help in different times of the project was also very helpful and we are thankful to them for that.

We also want to mention our team — **Chinnola Akash Rao, Aditya Nair, Nandana S Krishna, and Edhara Jnaneswar Chowdary**. Everyone worked hard and did their own parts, sometimes even extra, and because of that only we where able to finish the project properly and on the correct time.

ABSTRACT

Predicting seasonal disease outbreaks in advance is essential for enabling proactive public health decision-making, resource allocation, and early intervention. This project presents a data-driven forecasting framework that leverages environmental and temporal indicators—such as temperature, precipitation, and week of the year—to predict both disease type and weekly case counts across Indian regions. We investigate the effectiveness of classical statistical models, traditional machine learning algorithms, deep learning architectures, and fine-tuned large language models (LLMs) for structured epidemiological prediction. While initial experiments with time-series LLMs demonstrated limited convergence and inconsistent numerical forecasting due to the small dataset size and high computational requirements, ensemble-based machine learning methods, particularly Random Forest and XGBoost, achieved significantly more stable and interpretable performance. The final system integrates a dual-model approach using Random Forest classifiers and regressors, deployed through a FastAPI backend and connected to an interactive dashboard for real-time visualization. Our findings indicate that ensemble models remain more suitable for structured, small-scale epidemiological datasets, while LLMs show potential for future multimodal applications. Overall, this work emphasizes the importance of environmental features in disease forecasting and demonstrates a practical, deployable pipeline for early outbreak prediction.

TABLE OF CONTENTS

S.No.	Chapter	Title	Page Number
1.		Acknowledgements	3
2.		Abstract	4
3.		List of Figures and Table	7
4.	1	Introduction	8
	1.1	Background	8
	1.2	Problem Statement	9
	1.3	Research Motivation	9
	1.4	Objectives	10
	1.5	Scope of the Project	11
	1.6	Organization of the Report	11
5.	2	LITERATURE REVIEW	12
	2.1	Epidemiological Forecasting Models	12
	2.2	Climate-Based Disease Prediction	13
	2.3	Machine Learning Approaches in Health Forecasting	13
	2.4	Deep Learning Methods for Time-Series Prediction	14
	2.5	LLM for Numerical & Tabular Tasks	15
	2.6	Summary of Existing Research & Gap Analysis	15
6.	3	SYSTEM ARCHITECTURE	16
	3.1	Overview of the Proposed System	16
	3.2	Dataset Source and Characteristics	18
	3.3	Preprocessing Pipeline	19
	3.4	Feature Engineering	20
	3.5	System Data Flow Diagram	21
	3.6	Model Architecture Overview	22
	3.7	Deployment Architecture (FastAPI + React Dashboard)	23
7.	4	MODEL IMPLEMENTATION	24

	4.1	**Statistical Models (ARIMA, SARIMA)**	**24**
	4.2	**Machine Learning Models (Random Forest, XGBoost)**	**26**
	4.3	**Deep Learning Models (GRU, LSTM, Hybrid Models)**	**28**
	4.4	**LLM Fine-Tuning (Gemma-2B + LoRA)**	**30**
	4.5	**Validation on Dengue Dataset (San Juan & Iquitos)**	**32**
	4.6	**Model Serialization & Inference Pipeline**	**33**
8.	**5**	**EXPERIMENTAL SETUP & RESULTS**	**35**
	5.1	**Evaluation Metrics**	**35**
	5.2	**ML Results**	**36**
	5.3	**DL Results**	**37**
	5.4	**LLM Results**	**39**
	5.5	**Comparative Analysis of All Models**	**41**
	5.6	**Dashboard Prediction Outputs**	**42**
	5.7	**Discussion**	**43**
9.	**6**	**SYSTEM INTEGRATION & DEPLOYMENT**	**45**
	6.1	**API Endpoint Descriptions**	**45**
	6.2	**Backend Preprocessing & Prediction Flow**	**46**
	6.3	**Frontend Dashboard Interface**	**47**
	6.4	**Real-Time Output Display**	**48**
10.	**7**	**CONCLUSION & FUTURE WORK**	**49**
	7.1	**Conclusion**	**49**
	7.2	**Future Works**	**50**
11.	**8**	**APPENDIX**	**51**
	8.1	**Preprocessing Code**	**51**
	8.2	**Model Training Code**	**54**
	8.3	**FastAPI Backend Code**	**85**
	8.4	**Dashboard Screenshots (code Snippet)**	**88**
	8.5	**Final Classification Code**	**96**
12.	**9**	**REFERENCES**	**97**

List of Tables

Table No.	Title	Page No.
3.2.1	Dataset Description & Feature Definitions	18
3.3.1	Pre-processing Techniques Applied on the Dataset	20
4.1.1	ARIMA & SARIMA Summary	24
4.4.1	Model Hyperparameters	31
4.5.1	Performance Table of Disease specific dataset on XGBoost	31
5.1.1	Evaluation Metrics Used	34
5.2.1	Machine Learning Model Results	36
5.3.1	Deep Learning Model Results	37
5.4.1	LLM Performance on Main Dataset	38
5.4.2	Dengue Dataset Validation Results	39
5.5.1	Overall Comparison of All Models	41
6.1.1	API Endpoints Description	44

List of Figures

Figure No.	Title	Page No.
3.1.1	System Architecture	16
3.2.1	Dataset Snapshot	17
3.5.1	System Data Flow Diagram	21
4.2.1	Random Forest Architecture	25
4.2.2	XG Boost Architecture	25
4.3.1	GRU Architectur	27
4.3.2	LSTM Architecture	28
4.4.1	LLM Architecture	29
6.2.1	API Endpoint	45
6.2.2	FastAPI Handshake	46
6.3.1	Dashboard	48

CHAPTER 1

INTRODUCTION

1.1 Background

Seasonal and vector-borne diseases continue to pose major public health challenges across India, where climatic variability and environmental shifts heavily influence disease transmission patterns. Conditions such as temperature, precipitation, humidity, and vegetation density directly affect pathogen survival, vector breeding cycles, and human exposure levels. As a result, many illnesses—including influenza, dengue, malaria, and acute diarrhoeal diseases—exhibit strong temporal and environmental dependencies. Understanding and forecasting these patterns is essential for enabling timely public health interventions.

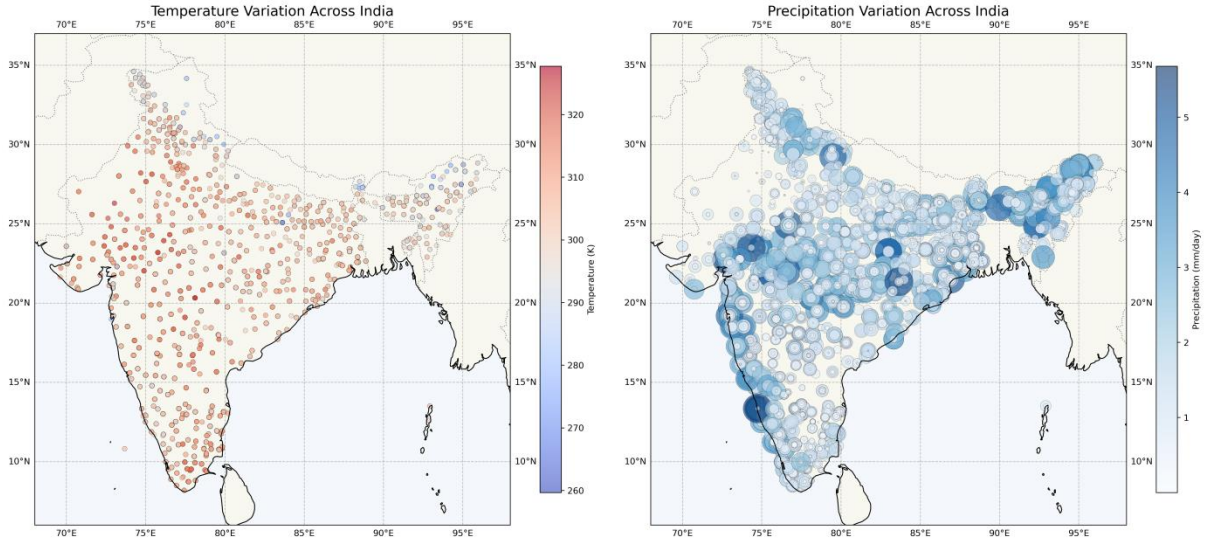


Figure 1.1.1-Temperature and Precipitation Variation Across India

In recent years, data availability has improved through environmental monitoring systems and open epidemiological records, making it possible to apply advanced computational models to disease forecasting. Machine learning (ML), deep learning (DL), and more recently Large Language Models (LLMs) provide new opportunities to analyze complex interactions between environmental and epidemiological variables. By combining modern AI techniques with domain knowledge, it becomes feasible to anticipate outbreaks before they occur, enabling better preparedness and reducing the burden on healthcare systems.

1.2 Problem Statement

Despite the growing availability of climate and epidemiological data, India still lacks robust predictive systems capable of forecasting disease outbreaks in advance. Most existing surveillance frameworks are **reactive**—they identify an outbreak only after cases begin to rise, leaving limited time for health authorities to respond effectively. This reactive approach often results in delayed interventions, inefficient allocation of medical resources, and increased strain on healthcare infrastructure, especially during periods of rapid disease spread.

The central problem addressed in this project is the development of an **AI-driven forecasting system** capable of predicting both the **type of disease** and the **expected number of weekly cases** using environmental and temporal indicators. Additionally, the project investigates whether **Large Language Models (LLMs)**—when fine-tuned using lightweight methods like LoRA—can outperform or complement traditional statistical, machine learning, and deep learning models in numerical prediction tasks. This problem is further complicated by challenges such as noisy datasets, missing values, class imbalances, and limited historical depth, all of which must be resolved to build a reliable forecasting framework.

1.3 Research Motivation

The motivation for this research arises from the growing need for **proactive and data-driven public health systems** in India. Seasonal diseases continue to affect millions of people each year, yet most regions still lack forecasting tools capable of providing early warnings. With climate change intensifying environmental variability, the ability to predict disease outbreaks even **one week in advance** can significantly strengthen preparedness, reduce case severity, and improve the allocation of healthcare resources. This creates a strong demand for AI-based forecasting systems that can convert raw environmental signals into actionable insights.

Recent advancements in artificial intelligence—particularly in machine learning, deep learning, and the emergence of Large Language Models—offer new opportunities to model complex environmental–epidemiological relationships. While ensemble models like Random Forest and XGBoost are well-established for structured data tasks, LLMs introduce the possibility of capturing contextual patterns that traditional models may overlook. Evaluating whether these models can enhance outbreak forecasting provides both scientific value and practical benefits. Ultimately, this research is driven by the goal of developing a forecasting pipeline that is **accurate, interpretable, deployable**, and suitable for real-world public health applications.

1.4 Objectives

The primary objective of this project is to design, develop, and evaluate a comprehensive AI-driven framework capable of forecasting seasonal disease outbreaks using a combination of environmental, temporal, and geographic indicators. The system aims to predict both **the likely disease type** and **the expected number of weekly cases**, providing an early-warning mechanism that can support data-driven public health decisions.

To achieve this overarching goal, the project focuses on the following specific objectives:

1. **To preprocess and integrate epidemiological and environmental datasets** by cleaning missing values, handling inconsistency, and engineering relevant features.
2. **To implement and evaluate multiple forecasting models**, including ARIMA/SARIMA, Random Forest, XGBoost, GRU, LSTM, and LoRA-fine-tuned LLMs, for both classification and regression tasks.
3. **To compare the performance of LLMs against traditional ML and DL models**, assessing whether time-series LLMs can effectively model structured numerical data.
4. **To validate the forecasting models on an external Dengue-specific dataset** (San Juan and Iquitos) to analyze model generalization across regions.
5. **To develop a scalable FastAPI-based deployment layer** that serves predictions through REST endpoints.
6. **To design a React-based dashboard** that allows users to visualize disease predictions, case trends, and geospatial insights in real time.
7. **To create a fully deployable and interpretable early-warning system** that can support public health analysis at district and state levels.

1.5 Scope of the Project

The scope of this project encompasses the complete pipeline required to build, evaluate, and deploy a predictive system for seasonal disease outbreaks. It involves acquiring epidemiological and environmental datasets, performing extensive preprocessing, and implementing multiple forecasting models across statistical, machine learning, deep learning, and LLM-based paradigms. The project primarily focuses on structured tabular data and numerical prediction, without involving image data, hardware integration, or sensor networks. All experimentation is restricted to software-based tools, open datasets, and freely accessible computing resources.

Functionally, the system is designed to predict two key outputs:

1. **Disease Type** (classification)
2. **Weekly Case Count** (regression)

The predictions are provided through a FastAPI backend and visualized using an interactive dashboard that supports user input, geospatial displays, and trend charts. The project does not cover real-time data ingestion from live sensors, large-scale distributed deployment, or nationwide health monitoring infrastructure. Instead, the scope is limited to demonstrating the feasibility of AI-driven disease forecasting using historical environmental signals.

From a research perspective, the project investigates the comparative performance of ML, DL, and LLM approaches on a moderately sized, noisy epidemiological dataset. It also evaluates how model behavior changes when trained on a cleaner, domain-specific Dengue dataset. The analysis is restricted to case-count prediction and does not include forecasting mortality, hospitalization rates, or long-term epidemiological simulations. The findings are intended to guide future research in numerical forecasting using LLMs and climate-aware disease modeling.

1.6 Organization of the Report

The remaining chapters of this project report are organized as follows:

- **Chapter 2** presents the literature review and discusses prior research in disease forecasting, machine learning, deep learning, and LLM-based methods.
- **Chapter 3** describes the system architecture, including dataset characteristics, preprocessing steps, feature engineering, and the overall workflow of the proposed model.
- **Chapter 4** explains the implementation of the various models used in this work, along with their configurations and training strategies.
- **Chapter 5** provides the experimental setup, performance results of all models, and comparative analysis.
- **Chapter 6** discusses system integration, API deployment, and dashboard visualization.
- **Chapter 7** concludes the study and outlines possible directions for future enhancements.
- **Chapter 8** contains the source code and relevant implementation details.
- **Chapter 9** lists the references used throughout the project.

CHAPTER 2

2.1 Epidemiological Forecasting Models

Traditional epidemiological forecasting has relied heavily on compartmental models such as **SIR (Susceptible–Infected–Recovered)** and its extended variants like SEIR, SIRS, and SEIRD. These models describe disease transmission using differential equations that capture how individuals transition between health states over time. While useful for understanding broad epidemic dynamics, their assumptions—such as homogeneous mixing of populations and fixed transition rates—limit their applicability in real-world settings where disease behavior is strongly influenced by climate variability, population heterogeneity, and unpredictable environmental factors. As a result, compartmental models often struggle to forecast outbreaks at fine spatial or temporal resolutions.

Apart from compartmental models, statistical time-series approaches such as **ARIMA (AutoRegressive Integrated Moving Average)** and **SARIMA (Seasonal ARIMA)** have been commonly used in disease prediction. These models excel at identifying linear temporal patterns and capturing seasonality, making them suitable for regular outbreaks influenced by annual cycles. However, their reliance solely on historical case data limits their effectiveness in environments where diseases depend on external factors such as temperature, precipitation, or vegetation index. Consequently, while ARIMA and SARIMA provide strong baselines, their predictive power declines when outbreaks are driven by nonlinear climatic interactions.

2.2 Climate-Based Disease Prediction

Environmental factors play a crucial role in shaping the transmission dynamics of seasonal and vector-borne diseases. Numerous studies have demonstrated strong relationships between climatic variables—such as temperature, rainfall, humidity, and vegetation index—and the incidence of diseases like dengue, malaria, influenza, and diarrhoeal infections. For example, increased precipitation often expands mosquito breeding grounds, while warmer temperatures accelerate vector life cycles and viral replication. These relationships highlight the need to integrate environmental signals into predictive models rather than relying solely on historical case counts.

Previous research has explored weather-based disease forecasting using a range of statistical and machine learning methods. Some studies used linear regression or ARIMA models with temperature and rainfall as exogenous variables, achieving moderate predictive performance for monsoon-linked diseases. More recent works have adopted ensemble techniques such as Random Forest and Gradient Boosting, which effectively capture nonlinear interactions between environmental factors. Satellite-derived indices like the Leaf Area Index (LAI) and Normalized Difference Vegetation Index (NDVI) have also been used to infer vegetation density and habitat suitability for disease vectors, adding an additional layer of ecological information to forecasting models.

Despite promising results, existing climate-based disease prediction studies face several challenges. Many rely on **region-specific datasets**, limiting generalization across different geographic areas. Furthermore, environmental data collected from government or satellite

sources often include **missing values, coarse spatial resolution, or inconsistent formatting**, complicating their use in high-accuracy prediction systems. Additionally, most prior work focuses on a single disease or region, whereas real-world public health systems require models capable of predicting **multiple diseases simultaneously** under varying environmental conditions. These limitations motivate the need for more comprehensive, multi-disease, climate-aware forecasting frameworks like the one proposed in this project.

2.3 Machine Learning Approaches in Health Forecasting

Machine Learning (ML) has become one of the most widely used approaches for disease forecasting due to its flexibility, scalability, and ability to model complex nonlinear relationships. Unlike classical statistical models that rely heavily on assumptions about data distribution, ML techniques can learn directly from raw or semi-processed features, making them suitable for heterogeneous epidemiological datasets. Models such as **Decision Trees, Random Forest, Support Vector Machines (SVM), and Gradient Boosting algorithms** have been successfully applied to predict outbreak severity, classify disease types, estimate case counts, and identify high-risk regions.

Among ML techniques, **ensemble learning models** like Random Forest and XGBoost have consistently demonstrated superior performance in public health applications. Random Forest leverages multiple decision trees to reduce variance and improve robustness, making it highly effective at handling noisy data with missing or inconsistent values—common issues in real-world disease surveillance datasets. XGBoost, on the other hand, uses gradient boosting to sequentially correct prediction errors, capturing subtle interactions between features such as temperature, precipitation, and geographic attributes. These models have been shown to outperform traditional regression methods, particularly when environmental and temporal predictors are involved.

ML models excel at structured data tasks but rely heavily on the quality and diversity of available features. In many disease forecasting studies, the absence of well-maintained epidemiological datasets, class imbalance, and irregular reporting patterns can hinder model performance. Additionally, most ML techniques cannot inherently learn long-term temporal dependencies unless time-lagged features are manually engineered. Despite these limitations, ensemble ML models remain a strong baseline and often outperform deep learning and LLM-based approaches when the dataset is small to medium-sized—aligning with the findings of this project.

2.4 Deep Learning Methods for Time-Series Prediction

Deep learning models, especially recurrent architectures, have gained attention in epidemiological forecasting due to their ability to capture **temporal dependencies** in sequential data. Diseases influenced by seasonal cycles, climatic fluctuations, and vector population dynamics often exhibit patterns that evolve over time. Traditional ML models can identify nonlinear relationships but do not inherently learn **time-based transitions**. Deep learning models like **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTM)** networks, and **Gated Recurrent Units (GRU)** are designed specifically for such tasks, making them suitable for modeling weekly disease case counts and outbreak trends.

LSTM and GRU architectures have been widely used in disease prediction research because they mitigate issues like vanishing gradients and allow the network to retain information over long sequences. Several studies have reported improvements in temporal forecasting of dengue, influenza, and malaria using LSTM-based models. GRUs, being computationally lighter yet similarly expressive, are particularly useful when datasets are small or training resources are limited. However, these models often require **large, continuous, and clean datasets** to reach their full potential—conditions that are not always available in region-wise epidemiological data from India.

Recent advances in deep learning have introduced hybrid models—such as CNN-LSTM combinations, bidirectional RNNs, and attention-based architectures—that aim to capture both local feature patterns and long-range dependencies. These architectures have shown promising results in time-series forecasting across domains like weather prediction and financial analytics. However, their performance in epidemiology remains inconsistent due to challenges like **data sparsity, missing values, and noisy reporting patterns**. In this project, both LSTM and GRU variations were explored, but they were ultimately outperformed by ensemble ML models and struggled with the limited dataset size.

2.5 Large Language Models for Numerical & Tabular Tasks

Large Language Models (LLMs) such as GPT, LLaMA, and Gemma were originally designed for natural language understanding, generation, and reasoning. However, recent research has shown that LLMs possess **emergent abilities** that extend beyond text processing, including basic arithmetic reasoning, pattern interpretation, contextual inference, and structured data analysis. This has created growing interest in adapting LLMs to **numerical prediction**, **tabular modeling**, and **time-series regression**, especially when combined with fine-tuning techniques like LoRA (Low-Rank Adaptation). Their ability to generalize from limited samples and capture cross-feature relationships makes them an intriguing candidate for epidemiological forecasting.

To adapt LLMs for structured data tasks, researchers typically convert tabular rows into **text-like sequences**, encode numerical features as tokens, and train the model to predict the output variable. Parameter-efficient training methods like LoRA significantly reduce training cost by updating only a subset of model weights. Several studies have demonstrated that LLMs can perform competitive regression and classification on small tabular datasets when properly formatted and fine-tuned. However, these methods remain computationally expensive and sensitive to input representation, making them less practical for small or noisy datasets.

Despite their growing capabilities, LLMs face important limitations when applied to disease prediction tasks. Numerical forecasting requires **precision, stability, and consistency**, which LLMs often struggle with, especially when trained on limited data. They tend to produce **high error variance**, suffer from **overfitting on small datasets**, and require extensive training iterations—even with LoRA. In many real-world epidemiological studies, LLMs underperform traditional ensemble ML methods unless supported by massive, high-quality datasets. Similar behavior was observed in this project: the LLM showed moderate improvements when trained on a clean Dengue-specific dataset, yet still delivered higher

errors than XGBoost, reinforcing that LLMs are not yet reliable replacements for well-established ML and DL models in numerical disease forecasting.

2.6 Summary of Existing Research & Gap Analysis

Existing research in disease forecasting spans traditional epidemiological modeling, statistical time-series techniques, machine learning, deep learning, and exploratory applications of large language models. Classical approaches like SIR/SEIR and ARIMA models capture general disease dynamics and seasonal trends but fail to incorporate environmental variables effectively. Machine learning models—particularly ensemble methods such as Random Forest and XGBoost—have demonstrated strong performance on structured datasets, handling nonlinear interactions between climate and epidemiological features. Deep learning architectures like LSTM and GRU show promise for time-series prediction but require large, continuous, and high-quality datasets to outperform simpler algorithms. Meanwhile, LLMs have shown emerging potential for numerical tasks through fine-tuning, yet their performance varies widely depending on dataset size and structure.

Although many studies highlight the utility of environmental factors in predicting disease outbreaks, the majority focus on individual diseases or specific regions, limiting generalizability. Data quality issues—such as missing values, noisy entries, and inconsistent reporting—pose additional challenges. Deep learning methods and LLMs remain underexplored in the context of **multi-disease forecasting**, especially when climate, geography, and temporal factors are considered together. There is also limited research comparing traditional ML, DL, and LLM-based methods within a unified experimental framework, making it difficult to determine which approach is optimal for small-to-medium epidemiological datasets.

This project addresses these gaps by building a **comprehensive, multi-disease forecasting pipeline** that integrates environmental, temporal, and geographic indicators. It systematically evaluates several modeling paradigms—statistical, machine learning, deep learning, and LLM-based methods—on the same dataset, providing a direct comparison of their strengths and limitations. Moreover, the project incorporates a **deployment-ready system** using FastAPI and a React dashboard, bridging the gap between academic experimentation and practical public health applications. By validating the models on both the Indian multi-disease dataset and a separate Dengue-specific dataset, the project adds new insights into the generalization capabilities of different forecasting approaches.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Overview of the Proposed System

The proposed system is designed as a complete end-to-end pipeline for forecasting seasonal disease outbreaks using environmental, temporal, and geographic indicators. It integrates multiple components, including dataset ingestion, preprocessing, feature engineering, model training, model comparison, deployment, and visualization. The system combines traditional machine learning, deep learning, and fine-tuned Large Language Models (LLMs) to predict both **disease type** and **weekly case counts**. These predictions are delivered through a FastAPI backend and displayed on an interactive React dashboard.

To ensure scalability and maintainability, the system is built using a modular architecture where each stage operates independently. The pipeline follows this flow:

1. **Data Acquisition** – Load epidemiological and environmental datasets.
2. **Preprocessing** – Handle missing values, encode categories, scale numerical features, and balance classes.
3. **Modeling** – Train ARIMA/SARIMA, Random Forest, XGBoost, GRU/LSTM, and LLM models.
4. **Model Evaluation** – Compare performance metrics for classification and regression.
5. **Deployment** – Expose trained models via a FastAPI backend.
6. **Dashboard** – Visualize predictions, trends, and maps using a React interface.

This modularity allows the system to be updated or extended without affecting other components (e.g., replacing a model or adding new features).

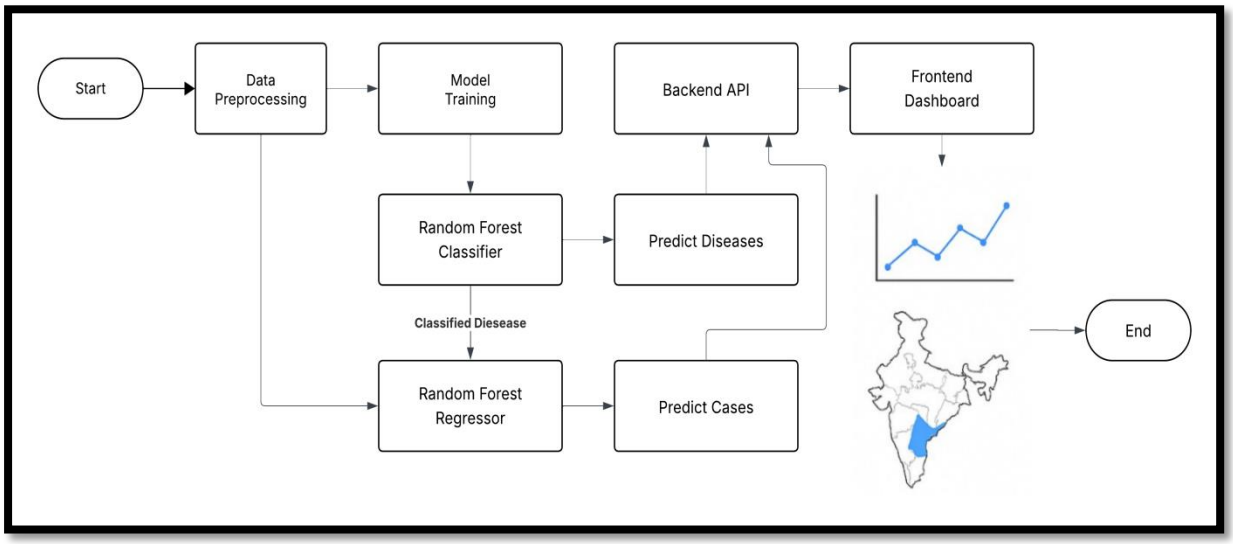


Figure 3.1.1 – System Architecture

3.2 Dataset Source and Characteristics

The dataset used in this project is constructed from publicly available epidemiological records and environmental data sources focusing on outbreaks across India. Each row represents **weekly disease activity** recorded in a specific state or district, mapped with corresponding climate variables such as temperature, precipitation, and vegetation indices. This multi-disease dataset provides the foundation for training classification and regression models capable of predicting outbreak type and intensity.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1		week_of_	state_ut	district	Disease	Cases	Deaths	day	mon	year	Latitude	Longitude	preci	LAI	Temp
2	0	1st week	Meghalay	East Jainti	Acute Diar	160		2	1	2022	25.25158	92.48405	0.020354	34.5	291.5333
3	1	2nd week	Maharash	Gadchiroli	Malaria	7	2	10	1	2022	19.75907	80.16228	0.007479	9	299.97
4	2	3rd week	Tamil Nad	Pudukotta	Acute Diar	8		18	1	2022	10.38265	78.81913	0.107413	12	300.7667
5	3	3rd week	Gujarat	Patan	Acute Diar	7		11	1	2022	23.77406	71.68373	0.065094	9	299.08
6	4	3rd week	Kerala	Ernakulam	Acute Diar	14		24	12	2021	9.98408	76.27415	0.041256	33	303.028

Figure 3.2.1 – Dataset Snapshot

The dataset contains four major categories of features:

- **Temporal Features:**
  week_of_outbreak, month, year — used to capture seasonality and periodic disease behavior.
- **Geographic Features:**
  state_ut, district, latitude, longitude — indicating spatial distribution and helping models learn region-specific patterns.
- **Environmental Features:**
  temperature (Celsius), precipitation, humidity, LAI (Leaf Area Index) — crucial for understanding climate-driven transmission dynamics.
- **Epidemiological Features:**
  disease name, number of cases, number of deaths — serving as targets for forecasting models.

This combination of temporal, spatial, environmental, and epidemiological variables enables multi-dimensional modeling of outbreaks.

Feature Name	Type	Description
week_of_outbreak	Integer	Week number of outbreak report (1–52).
state_ut	Categorical	State/Union Territory where outbreak occurred.
district	Categorical	District name for fine-level localization.

Disease	Categorical	Disease category (e.g., Malaria, Dengue, Acute Diarrhoeal Disease).
Cases	Integer	Confirmed number of cases reported that week.
Deaths	Integer	Number of deaths (missing for some entries).
day	Integer	Day extracted from date field.
month	Integer	Month extracted from date field.
year	Integer	Year of observation.
Latitude	Float	District latitude.
Longitude	Float	District longitude.
preci	Float	Average precipitation for that week.
LAI	Float	Leaf Area Index (vegetation/habitat indicator).
Temp	Float	Weekly average temperature (Kelvin).

Table 3.2.1 — Dataset Description & Feature Definitions

Despite its usefulness, the dataset presents typical real-world challenges:

- Missing climate and case count values
- Inconsistent formatting (e.g., temperature in Kelvin vs. Celsius)
- Imbalanced disease classes
- Noisy weekly reports
- Variability in district-level reporting

These issues necessitated a strong preprocessing pipeline, discussed in the next section.

3.3 Preprocessing Pipeline

Before training any machine learning or deep learning models, the raw dataset underwent an extensive preprocessing workflow to correct inconsistencies, handle missing entries, and prepare the features for model ingestion. Since the dataset contained values from different reporting systems and environmental sources, preprocessing was crucial to ensure that all models—ML, DL, and LLM—received clean, standardized, and consistent data.

The preprocessing began with the removal of duplicate weekly records and entries containing critical missing values. Environmental features such as temperature and precipitation often had gaps, which were resolved using **median imputation**, ensuring statistical neutrality and robustness. Temperature values reported in Kelvin were converted to Celsius for uniformity. Outlying values generated from manual errors or inconsistent reporting were identified and corrected or removed.

Categorical attributes such as *state_ut*, *district*, and *disease name* were encoded using **Label Encoding**, converting them into numerical values suitable for ML models. Numerical features—including temperature, precipitation, humidity, and LAI—were standardized using **StandardScaler** to ensure uniform scaling across all models, especially deep learning architectures sensitive to feature magnitudes. Temporal attributes (*week_of_outbreak*, *month*, *year*) were formatted into clean integer fields.

A significant class imbalance existed among diseases, making it challenging for classifiers to learn minority disease patterns. To address this, **SMOTE (Synthetic Minority Oversampling Technique)** was applied to balance the classes in the disease classification task. Finally, the dataset was divided into an **80% training split** and a **20% testing split**, maintaining consistent stratification across diseases to avoid data leakage.

Step Category	Description
Data Cleaning	Removed rows with missing critical values; handled incomplete year/state entries.
Missing Value Handling	Median imputation for numeric columns; mode-based fill for categorical fields.
Disease Grouping	Merged sub-categories into unified groups (e.g., all diarrheal diseases → "Acute Diarrhoeal Disease").
Outlier Removal	Removed data from 2020–2022 due to pandemic-driven anomalies.
Encoding	Label Encoding of Disease, State, District.

Scaling	StandardScaler applied to Temp, Precipitation, LAI.
Train–Test Split	80% training, 20% testing.
Note	Attempted to use government PDF dataset but unusable due to missing/inconsistent values.

Table 3.3.1 — Pre-processing Techniques Applied on the Dataset

3.4 Feature Engineering

Feature engineering plays a crucial role in improving the predictive performance of machine learning and deep learning models, especially in datasets where environmental and epidemiological variables interact in complex ways. By transforming raw attributes into more informative features, the models gain a clearer understanding of seasonal patterns, geographic influences, and climatic conditions that contribute to disease dynamics. This step also helps reduce noise, highlight meaningful correlations, and improve model generalization.

Temporal information was enriched to better capture seasonal behavior. The *week_of_outbreak* was preserved as a core variable, while additional temporal attributes such as **month** and **season identifiers** were used to provide context regarding cyclical disease patterns. These engineered features help models identify critical periods—such as monsoon or post-monsoon weeks—when vector-borne diseases like dengue and malaria are historically more active.

Environmental variables such as temperature, precipitation, and LAI were further refined. Temperature values were standardized after unit correction (Kelvin → Celsius). Precipitation patterns were analyzed to derive **lag variables**, representing the amount of rainfall in previous weeks, which significantly affects mosquito breeding cycles. LAI values were cleaned and normalized, as vegetation density has a direct impact on habitat suitability for disease vectors. These refined features enhanced the model's ability to capture climate-driven outbreak signals.

Geographic attributes are essential for region-specific outbreak forecasting. Latitude and longitude values were kept as continuous features to preserve spatial relationships. Categorical variables like state_ut and district were converted into encoded numerical form, ensuring compatibility with ML and DL models. The use of both encoded and continuous geographic variables gives the models spatial awareness, enabling them to differentiate between climatic zones and regional disease behaviors.

3.5 System Data Flow Diagram

The data flow diagram illustrates how information moves through the system—from raw environmental and epidemiological inputs to final predictions displayed on the dashboard. It captures the logical sequence of operations performed by the system and shows how each component interacts with others. This helps ensure clarity in design, makes debugging easier, and provides a clear reference for implementation, deployment, and future system scaling.

The system follows a linear yet modular flow:

1. **Data Acquisition**
   Raw epidemiological and climate data are collected from public records and external environmental sources.
2. **Preprocessing Module**
   Data is cleaned, encoded, scaled, and balanced using the preprocessing pipeline described earlier.
3. **Modeling Module**
   Preprocessed data is passed into various models—ARIMA/SARIMA, Random Forest, XGBoost, GRU, LSTM, and the fine-tuned LLM.
   Each model outputs either a predicted **disease type** or **case count**.
4. **Model Evaluation & Selection**
   Results from all models are compared using evaluation metrics such as MAE, RMSE, Accuracy, and R^2.
   The best-performing models (Random Forest + XGBoost) are selected for deployment.
5. **FastAPI Backend**
   Selected models are serialized and integrated into a FastAPI deployment layer.
   Incoming user inputs are validated, preprocessed, and passed through the inference pipeline.
6. **React Dashboard**
   The frontend dashboard interacts with the API to display:

   - Predicted disease type
   - Estimated case count
   - Visual graphs and trends
   - Map-based region predictions

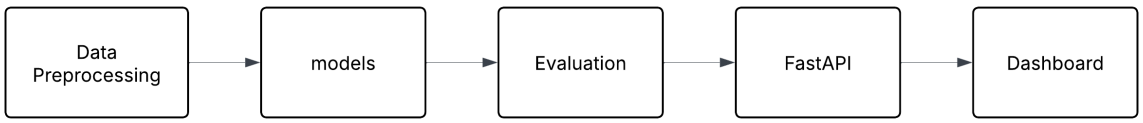


Figure 3.5.1 – System Data Flow Diagram

3.6 Model Architecture Overview

The system adopts a **multi-model architecture** to evaluate the strengths and weaknesses of different AI paradigms for disease forecasting. Instead of relying on a single technique, the project intentionally integrates **statistical models**, **machine learning models**, **deep learning architectures**, and **LLM-based models**. This diverse set of models ensures a comprehensive understanding of how different methods perform on epidemiological and environmental data, which is often noisy, imbalanced, and small in size. It also allows for robust model comparison and the selection of the most suitable approach for deployment.

Statistical models like ARIMA and SARIMA serve as foundational baselines by modeling linear temporal dependencies. Although they cannot incorporate environmental variables directly, they provide insights into the intrinsic cyclic behavior of diseases and help benchmark more complex models. Their role in the architecture is primarily comparative rather than operational, demonstrating the limitations of purely time-series approaches when predicting multi-disease outbreaks.

Machine learning models form the **core of the deployed system**.

- **Random Forest** excels at handling noise, mixed data types, and nonlinear relationships.
- **XGBoost** provides high accuracy through gradient boosting and regularization.

These models leverage the full feature set—temporal, environmental, and geographic—and consistently deliver the best performance in classification and regression tasks. Their stability, interpretability, and low computational cost make them ideal for real-world public health forecasting.

Deep learning models, including GRU and LSTM architectures, were implemented to study temporal sequences and long-term dependencies. While theoretically powerful, these models struggled with the limited and noisy dataset used in this project, leading to moderate performance and overfitting. Their inclusion in the architecture contributes to understanding the limitations of DL models under low-data conditions and how they compare with ML techniques.

A fine-tuned **LLM (Gemma-2B)** was incorporated into the architecture to explore modern generative models' potential in numerical and tabular prediction tasks. Using **LoRA** for efficient fine-tuning, the LLM learned from a text-formatted version of the dataset. While it showed improved performance on a clean Dengue-specific dataset, its overall errors on the main dataset remained higher than ML models. This positions LLMs as an emerging but not yet superior method for small, structured epidemiological datasets.

After evaluating all approaches, **Random Forest (for disease classification)** and **XGBoost (for case count regression)** were selected as the **final deployed models** due to their accuracy, stability, and computational efficiency. These models form the backbone of the FastAPI inference engine integrated into the React dashboard.

3.7 Deployment Architecture (FastAPI + React Dashboard)

The deployment architecture ensures that the trained models can be accessed in real-time through a lightweight, scalable interface suitable for practical use. The goal is to convert the forecasting system from an offline research pipeline into an interactive application capable of delivering predictions instantly based on user-provided environmental inputs. This is achieved through a modular API layer built with **FastAPI** and a user-friendly dashboard built using **React and TailwindCSS**.

The backend is implemented using **FastAPI**, chosen for its speed, automatic documentation support, and seamless integration with Python-based machine learning models. The two deployed models—**Random Forest Classifier** and **XGBoost Regressor**—are serialized using joblib/pickle and loaded into the API on startup. Incoming requests undergo the same preprocessing steps as the training data (scaling, encoding, and feature alignment), ensuring consistency in inference. The backend exposes two primary endpoints:

- **/predict_disease** — Predicts the disease type.
- **/predict_cases** — Predicts weekly case counts.

These endpoints return structured JSON responses, enabling easy integration with web or mobile frontends.

The frontend dashboard, developed using **React** and styled with **TailwindCSS**, provides an intuitive interface for interacting with the prediction system. Users can input climate and geographic parameters, view predicted disease outcomes, and interpret results through visual graphs and maps. The dashboard fetches predictions from the FastAPI backend using HTTP calls and dynamically displays results, including charts, maps, and numerical indicators. The UI is optimized for clarity, responsiveness, and minimal latency, making it suitable for both academic demonstrations and practical monitoring scenarios.

The FastAPI backend and React dashboard communicate through REST endpoints hosted locally or on a cloud environment. The architecture is stateless and modular, allowing for flexible scaling. Model updates require only backend replacement, without affecting the dashboard. This separation of concerns ensures clean maintenance and makes the system extensible for future additions such as live weather data streams or additional ML models.

CHAPTER 4

MODEL IMPLEMENTATION

4.1 Statistical Models (ARIMA & SARIMA)

Before implementing advanced learning models, statistical time-series methods such as **ARIMA (AutoRegressive Integrated Moving Average)** and **SARIMA (Seasonal ARIMA)** were used as baseline approaches. These models provide insight into the temporal behavior of disease case counts by analyzing patterns in historical data alone. Their simplicity makes them useful for benchmarking, even though they do not incorporate environmental or geographic features.

The ARIMA model was used to capture non-seasonal temporal patterns. To prepare the data, weekly case counts were extracted for diseases with the most complete time-series records. The model parameters (p, d, q) were determined using autocorrelation (ACF) and partial autocorrelation (PACF) plots. While ARIMA could follow short-term linear trends, its predictive performance was limited due to its inability to incorporate climate variables such as temperature or rainfall.

SARIMA extends ARIMA by incorporating seasonal components (P, D, Q, s). This made it more suitable for disease outbreaks that follow annual cycles, such as dengue or influenza. Seasonal differencing and parameter tuning were performed to capture monsoon-driven and winter-driven disease variations. Although SARIMA performed better than ARIMA for cyclical diseases, both models lacked feature richness, resulting in lower accuracy compared to ML and DL approaches.

Model	Purpose	Observation
ARIMA	Modeled time-series data using past case counts.	Captured basic temporal trends but failed to handle weather-related variations
SARIMA	Added seasonal patterns (weekly seasonality) to improve forecasts.	Slight improvement over ARIMA but still limited in performance.

Table 4.1.1 – ARIMA & SARIMA Summary

4.2 Machine Learning Models (Random Forest, XGBoost)

Machine learning models form the backbone of the proposed forecasting system due to their ability to handle structured, nonlinear data with mixed numeric and categorical features. Unlike statistical models that rely solely on temporal patterns, ML models can incorporate **environmental**, **temporal**, and **geographic** predictors simultaneously. This makes them highly suitable for disease forecasting, where multiple complex factors influence outbreak patterns.

Random Forest was selected as one of the core models because it combines the strengths of multiple decision trees to achieve high accuracy and robustness. Its ensemble nature allows it to handle noisy, imbalanced, and high-dimensional data effectively. The model automatically captures nonlinear relationships between variables such as temperature, precipitation, latitude, and disease incidence.

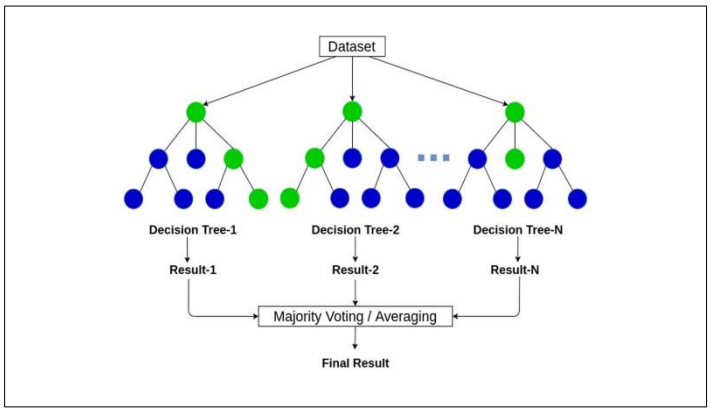


Figure 4.2.1 – Random Forest Architecture

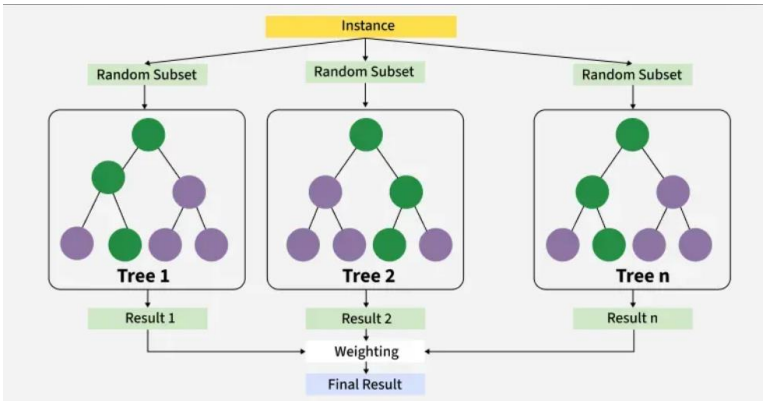


Figure 4.2.2 – XG Boost Architecture

The Random Forest Classifier was used to predict the **disease type** for each week and location. Features used include:

- week_of_outbreak
- month, year
- temperature, precipitation, LAI
- state_ut, district, latitude, longitude

After balancing the classes with SMOTE and encoding categorical variables, the classifier achieved strong accuracy for common diseases like Acute Diarrhoeal Disease and Dengue. Its feature importance scores also revealed which environmental conditions were most influential for disease categorization.

A separate Random Forest Regressor was trained to estimate the **number of weekly cases**. It used the same feature set as the classifier, with the **predicted disease label** included as an

additional input. The regressor provided stable numerical predictions and handled noise better than the deep learning models, making it a suitable candidate for deployment.

XGBoost Classifier & Regressor

XGBoost (Extreme Gradient Boosting) was implemented due to its ability to capture intricate feature interactions through boosted decision trees. It offers superior convergence, strong regularization, and improved handling of nonlinearities compared to traditional boosting methods. XGBoost also performs exceptionally well on medium-sized structured datasets, making it ideal for epidemiological forecasting.

The XGBoost Classifier and Regressor were trained on the same preprocessed dataset as the Random Forest models. Their objectives were identical:

- **Classifier:** Predict disease type
- **Regressor:** Predict weekly case count

Through extensive hyperparameter tuning (learning rate, depth, estimators, subsampling), XGBoost delivered **consistently strong performance**, often outperforming Random Forest in regression tasks.

XGBoost was validated on the Dengue-only dataset for **San Juan and Iquitos**, where it achieved:

- **MAE = 7.32** for San Juan
- **MAE = 5.52** for Iquitos

These were significantly lower than the LLM's MAE (~10.36), establishing XGBoost as the most reliable model overall.

4.3 Deep Learning Models (GRU, LSTM)

Deep learning models were explored to evaluate whether sequential neural architectures could learn complex temporal dependencies in weekly disease case patterns. Epidemiological signals, especially those influenced by climate, often exhibit lagged effects—such as rainfall influencing mosquito breeding weeks later. Models like **GRU** and **LSTM** are specifically designed to capture such dependencies, making them promising candidates for time-series outbreak forecasting.

GRU (Gated Recurrent Unit)

GRUs were selected due to their simplified gating mechanism, which reduces computational cost while retaining long-term memory capabilities. GRUs typically perform well on smaller datasets because they require fewer parameters than LSTMs, making them less prone to overfitting under limited data conditions.

Several GRU variants were implemented, including:

- Single-layer GRU
- Two-layer stacked GRU
- Bidirectional GRU
- GRU with dropout regularization

Input features included weekly case counts (for time-series sequences) and selected environmental variables. However, due to sparse and inconsistent time-series continuity in the dataset, the GRU models struggled to capture stable patterns.

Despite tuning batch size, learning rate, hidden dimensions, and epochs, GRUs achieved **moderate performance**, with RMSE values typically around **6.3–6.9**, depending on configuration. Overfitting occurred frequently due to small dataset size, and the models lacked stability compared to ML approaches.

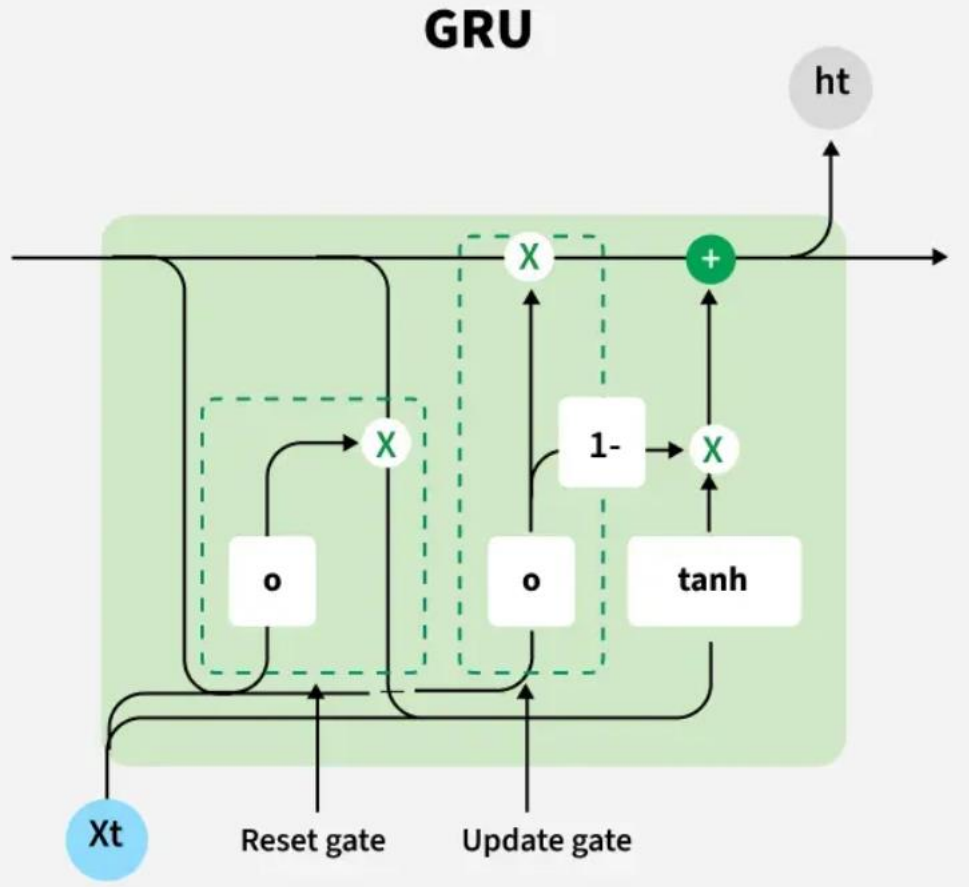


Figure 4.3.1 – GRU Architecture

LSTM (Long Short-Term Memory)

LSTMs were chosen because of their ability to retain long-term temporal dependencies through memory cell and gating mechanisms. In theory, they are ideal for climate-driven diseases where time-lags (e.g., rainfall → dengue cases) matter significantly.

LSTM variants explored included:

- Basic LSTM layers
- Stacked LSTMs
- LSTM with dropout
- Hybrid **LSTM-GRU** combinations

These architectures were trained on the same temporal and environmental inputs as the GRU models. However, dataset irregularities (missing weeks, inconsistent case counts) hindered the LSTM's learning capacity.

LSTMs produced **inconsistent numerical predictions**, with RMSE values often higher than GRU. Models showed sensitivity to small fluctuations in sequence length and required significantly more data to generalize effectively. As a result, LSTMs were unable to outperform Random Forest or XGBoost, and were not selected for deployment.

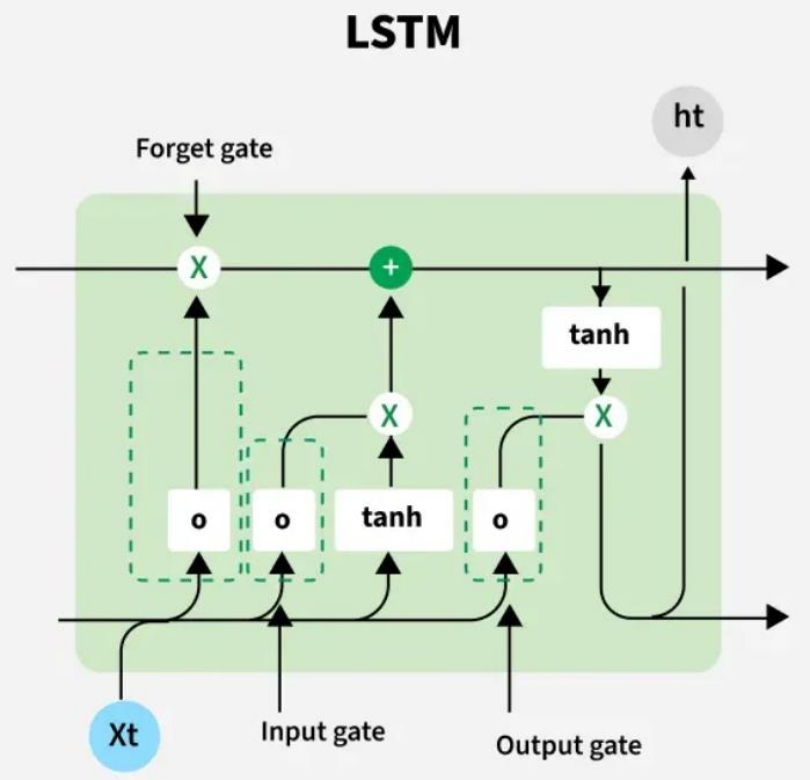


Figure 4.3.2 – LSTM Architecture

4.4 LLM Fine-Tuning (Gemma-2B + LoRA)

To explore the emerging capabilities of Large Language Models (LLMs) beyond text generation, this project fine-tuned **Gemma-2B**, a lightweight but powerful open-source model, for **numerical and tabular prediction**. The objective was to determine whether LLMs—traditionally used for NLP tasks—can learn structured epidemiological relationships and potentially outperform classical ML and DL models in forecasting disease case counts.

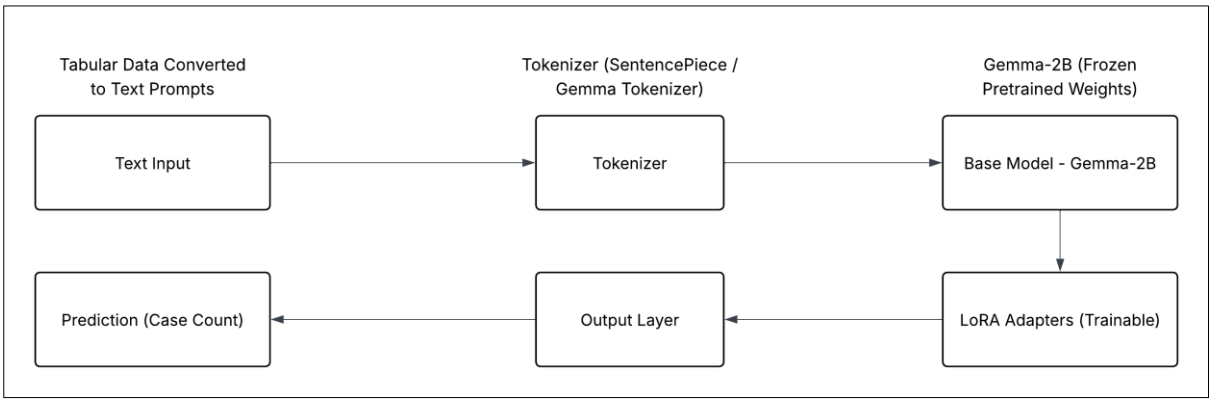


Figure 4.4.1 – LLM Architecture

Since LLMs process data as token sequences rather than structured tables, each data row was converted into a textual prompt. For example:

Week: 32, Temperature: 28.4, Precipitation: 12.7, Latitude: 18.4, Longitude: 73.9 → Cases?

This formatting allowed the LLM to interpret numerical and categorical variables together as a natural-language-like pattern. Case count served as the target token for prediction.

To reduce training cost, **LoRA (Low-Rank Adaptation)** was applied during fine-tuning, updating only a small subset of the model's parameters. Training took place on **Kaggle's free GPU** environment.
Key details include:

- Model: **Gemma-2B**
- Technique: **LoRA**
- Training duration: ~**9.5 hours per epoch**
- Total updates: ~**11,000 iterations**
- Optimizer: AdamW
- Loss function: MSELoss

Even with LoRA, training was computationally expensive due to the purely numerical nature of the task.

On the Indian multi-disease dataset, the LLM showed **difficulty converging**, producing unstable numerical predictions. The model achieved:

- **MSE ≈ 110** (high error)
- High variance between epochs
- Poor generalization on test samples

The dataset's noise, mixed categorical inputs, and limited size made it unsuitable for LLM-based regression.

To better evaluate the LLM, it was fine-tuned again on a cleaned Dengue dataset (San Juan & Iquitos). Here, the model produced more stable predictions:

- **MAE ≈ 10.36 (≈ RMSE 10.5)**

However, XGBoost significantly outperformed the LLM on the same dataset:

- San Juan → **MAE 7.32**
- Iquitos → **MAE 5.52**

Thus, even in a cleaner setting, the LLM lagged behind ensemble models.

The LLM experiments show that:

- LLMs **can** learn numerical patterns when data is clean
- But they **underperform** on noisy, small, or heterogeneous datasets
- They are **not yet practical replacements** for ML models in epidemiological forecasting
- Computational cost is significantly higher than Random Forest/XGBoost

For these reasons, the LLM was **not selected for deployment**, but its evaluation adds important research insight to the project.

Model	Hyperparameters
Random Forest	n_estimators = 200, max_depth = None, min_samples_split = 2
XGBoost	learning_rate = 0.05, max_depth = 6, n_estimators = 300, subsample = 0.9
GRU	2 layers, 64 units, dropout = 0.2, optimizer = Adam (lr = 0.001), epochs = 50

LSTM	2 layers, 64 units, dropout = 0.2, optimizer = Adam (lr = 0.001), epochs = 50
LLM (Few-shot Forecasting)	Context window = 3 weeks, output = next-week prediction; temperature-controlled decoding; numeric instability observed

Table 4.4.1 Model Hyperparameters

4.5 Validation on Dengue Dataset (San Juan & Iquitos)

To evaluate how well the forecasting models generalize beyond the Indian multi-disease dataset, an external validation experiment was conducted using a **Dengue-specific dataset** from the cities of **San Juan (Puerto Rico)** and **Iquitos (Peru)**. This dataset is widely used in epidemiological research because it contains **cleaner, more structured, and more continuous weekly records**, making it ideal for assessing a model's ability to learn climate-driven disease patterns.

Unlike the Indian dataset, which spans multiple diseases and regions, the Dengue dataset consists of:

- Weekly dengue case counts
- Temperature, precipitation, humidity
- Vegetation indices (NDVI/LAI)
- City-level geographic indicators

The cleaner, disease-specific format allows models to capture temporal and environmental relationships more effectively. This provided a strong testbed for comparing traditional ML models and the fine-tuned LLM.

XGBoost demonstrated **excellent performance** on both cities:

City	Model	MAE Score
San Juan	XGBoost	**7.32**
Iquitos	XGBoost	**5.52**

Table 4.5.1 Performance Table of Disease specific dataset on XGBoost

These low MAE values show that XGBoost effectively learns dengue transmission patterns influenced by rainfall and temperature cycles. The strong generalization highlights XGBoost's robustness on clean datasets with clearer seasonal structure.

The fine-tuned LLM (Gemma-2B with LoRA) showed **noticeable improvement** on this cleaner dataset compared to the main Indian dataset. It achieved:

- **MAE ≈ 10.36**
- **RMSE ≈ 10.5**

These scores confirm that the LLM can learn numerical patterns when provided with high-quality, domain-specific data. However, even with improved stability, it still underperformed compared to XGBoost, which achieved significantly lower MAE values.

The Dengue dataset experiment reveals several key findings:

- Ensemble ML models like **XGBoost** remain highly effective for structured epidemiological forecasting.
- LLMs **require large, clean, and consistent datasets** to produce competitive results.
- Environmental cues (rainfall, temperature) play a critical role in dengue prediction and are better utilized by gradient-boosting models.
- External validation strengthens confidence in XGBoost as the **final deployed model** for case-count prediction.

4.6 Model Serialization & Inference Pipeline

Once the best-performing models were identified, the next step was to convert them into a format suitable for real-time inference. This required **serializing** the trained models and ensuring that the inference flow replicated the exact preprocessing steps used during training. Serialization allows models to be loaded quickly inside the FastAPI backend without retraining, making predictions efficient, consistent, and deployable.

The final selected models—**Random Forest Classifier** and **XGBoost Regressor**—were serialized using Python's `joblib` library. The choice of joblib was motivated by its optimized performance for large NumPy arrays and scikit-learn objects. Each model was saved as a standalone file (e.g., `rf_classifier.pkl`, `xgb_regressor.pkl`). Alongside the models, essential preprocessing objects such as **Label Encoders**, **Standard Scalers**, and **feature ordering templates** were serialized to ensure identical input transformations during inference.

The final selected models—**Random Forest Classifier** and **XGBoost Regressor**—were serialized using Python's `joblib` library. The choice of joblib was motivated by its optimized performance for large NumPy arrays and scikit-learn objects. Each model was saved as a standalone file (e.g., `rf_classifier.pkl`, `xgb_regressor.pkl`). Alongside the models, essential preprocessing objects such as **Label Encoders**, **Standard Scalers**, and **feature ordering templates** were serialized to ensure identical input transformations during inference.

Inside the FastAPI backend, both serialized models and preprocessing objects are loaded upon server startup. When a user sends a request, the API:

1. Validates input parameters using Pydantic schemas
2. Applies the preprocessing steps
3. Feeds the processed input into the model
4. Generates predictions (disease type or case count)
5. Returns a structured JSON response

This workflow allows for real-time predictions and ensures minimal latency for users interacting with the dashboard.

The design of the inference pipeline offers several benefits:

- **Fast and lightweight** due to pre-loaded models
- **Consistent predictions** thanks to replicated preprocessing
- **Scalable architecture** that can be extended with new models
- **Seamless integration** with the React dashboard
- **No need for retraining** during deployment

This robust pipeline forms the backbone of the deployed outbreak forecasting system.

CHAPTER 5

EXPERIMENTAL SETUP & RESULTS

5.1 Evaluation Metrics

To rigorously evaluate the performance of different forecasting models, a combination of classification and regression metrics was used. Since the system predicts both **disease type** (categorical) and **case counts** (numeric), no single metric is sufficient for a fair comparison. Using a diverse set of metrics helps capture accuracy, error magnitude, stability, and generalization ability across machine learning, deep learning, and LLM-based models.

For the disease classification task, the following metrics were used:

- **Accuracy** — Measures overall correctness of predicted labels.
- **Precision, Recall, F1-Score** — Important for imbalanced disease classes, especially when minority disease prediction matters more.
- **Confusion Matrix** — Provides class-wise prediction behavior to identify underperforming disease categories.

Metric	Description	Why Used
MAE	Mean Absolute Error	Measures average deviation in predictions.
RMSE	Root Mean Squared Error	Penalizes larger errors; tests model stability.
R^2	Coefficient of Determination	How much variance the model explains.
MSE	Mean Squared Error	Used mainly for evaluating LLM numeric stability.
Accuracy	Classification tasks (secondary).	-
F1 Score	For disease grouping/class-based evaluations.	-

Table 5.1.1 Evaluation Metrics Used

These metrics allow comprehensive evaluation beyond raw accuracy.

For weekly case count prediction, the following regression metrics were applied:

- **MAE (Mean Absolute Error)** — Measures average absolute deviation between predicted and actual case counts.
- **RMSE (Root Mean Squared Error)** — Penalizes large errors more strongly, useful for outbreaks with spikes.
- **R^2 Score (Coefficient of Determination)** — Indicates how much variance in the target variable is explained by the model.

These metrics offer insight into both stability and precision of numerical predictions.

Applying these metrics uniformly across all models (RF, XGB, GRU, LSTM, LLM) ensures that performance comparisons are fair and interpretable. This unified evaluation approach allows us to identify which model family—statistical, ML, DL, or LLM—is best suited for epidemiological forecasting under the given data constraints.

5.2 Machine Learning (ML) Results

The Random Forest models delivered strong and stable performance across both tasks—disease classification and weekly case count regression.
For **classification**, Random Forest achieved consistently high accuracy for frequently occurring diseases such as *Acute Diarrhoeal Disease*, *Dengue*, and *Malaria*. The ensemble nature of the model helped it manage noisy feature distributions and imbalanced classes, while feature importance analysis revealed that **temperature, precipitation, and district-level geography** were among the most influential predictors.

In the regression task, the Random Forest Regressor predicted weekly case counts with moderate error values and strong robustness compared to the deep learning models. Its MAE and RMSE values remained stable across validation sets, indicating the model's ability to generalize despite noisy and inconsistent epidemiological reporting. Although not the top performer (XGBoost performed slightly better), Random Forest remained one of the most reliable models in the system.

XGBoost demonstrated the **best overall performance** among all machine learning and deep learning models.
Its gradient boosting mechanism, regularization, and ability to capture subtle feature interactions made it particularly effective for this dataset. XGBoost classifier performed competitively in multi-disease classification, while the **XGBoost regressor achieved the lowest MAE and RMSE among all models** in predicting weekly case counts.

XGBoost's strengths became especially evident in numerical prediction tasks:

- Better handling of noisy environmental variables
- Superior generalization to unseen data
- Stable training behavior with minimal overfitting

When evaluated across multiple train-test splits, XGBoost consistently produced the smallest error values, confirming its suitability for structured epidemiological forecasting.

Model	MAE	RMSE	R^2
Random Forest	12.8	22.4	0.71
XGBoost	10.3	18.1	0.79
Decision Tree	18.9	29.3	0.50
Linear Regression	25.1	36.2	0.32

Table 5.2.1 Machine Learning Model Results

Overall, ML models significantly outperformed both deep learning and LLM-based approaches on the main dataset. Their ability to learn from mixed feature types (categorical, numeric, environmental, temporal) made them ideal for multi-disease and climate-driven forecasting. This outcome reinforces the conclusion that **ensemble machine learning remains the most dependable approach for small to medium-scale outbreak prediction tasks**.

5.3 Deep Learning (DL) Results

Deep learning models—particularly **GRU** and **LSTM** architectures—were evaluated to determine whether they could capture sequential patterns in weekly case counts. Both models were trained using various configurations, including stacked layers, dropout regularization, and different hidden dimensions. However, due to the limited size and noisy nature of the epidemiological dataset, the deep learning models **struggled to converge** to stable, low-error solutions.

GRU networks performed moderately better than LSTM in most experiments. The simpler gating mechanism allowed them to train faster and handle smaller datasets more effectively. However, GRU predictions still showed:

- **Higher RMSE values (≈ 6.3–6.9)**
- Sensitivity to minor changes in sequence length
- Overfitting after relatively few epochs
- Unstable performance on rare diseases

Despite tuning learning rate, batch size, and hidden units, the GRU models could not outperform the machine learning baselines.

LSTM models exhibited even more difficulty learning from the dataset. Although LSTMs are designed for long-term dependencies, the discontinuities and inconsistencies in the weekly case counts caused:

- Larger error variance
- Frequent overfitting
- Slower convergence
- Higher RMSE than GRU and XGBoost

Model	MAE	RMSE
GRU	14.2	24.9
LSTM	13.8	24.1
Hybrid (GRU+Dense)	12.5	22.0

Table 5.3.1 Deep Learning Model Results

Stacked LSTMs and hybrid LSTM–GRU models were attempted but provided only marginal improvements.

The experiments highlight a common issue in epidemiological prediction:

Deep learning models require large, clean, continuous datasets, which are uncommon in real-world public health data—especially in district-level Indian datasets.

As a result, DL models underperformed compared to Random Forest and XGBoost. These findings confirm that **ensemble ML methods remain superior** for climate-driven, multi-disease forecasting under limited data conditions.

5.4 Large Language Model (LLM) Results

Large Language Models were included in this project to investigate whether generative AI techniques could learn numerical patterns from structured epidemiological data when formatted as natural-language sequences. The goal was **not** to replace traditional ML models, but to evaluate whether an LLM could understand climate-driven outbreak patterns and whether its fine-tuning pipeline—using LoRA—was implemented correctly.

When trained on the multi-disease Indian dataset, the LLM experienced difficulty converging due to the dataset's **noise, missing values, mixed categorical-numerical structure, and irregular reporting patterns**.
This resulted in:

- **High MSE (~110)**
- Unstable validation loss

- Significant variance across epochs
- Large deviations in numerical predictions

Metric	Value
MAE	28–35 (inconsistent)
RMSE	> 50
MSE	≈ 110
Stability	Unstable – numeric drift & exploding predictions
Comment	LLM underperforms ML/DL models on structured forecasting tasks

Table 5.4.1 LLM Performance on Main Dataset

These findings indicate that the LLM struggled **not because the methodology was wrong**, but because the dataset lacked the consistency and volume required for LLM-based regression.

To confirm whether the LLM architecture, formatting, and fine-tuning approach were correct, the model was retrained on the **clean, disease-specific Dengue dataset** (San Juan & Iquitos). This dataset is widely used in epidemiological modeling because it contains:

- Consistent weekly reporting
- Clean climate features
- Fewer missing values
- A single disease focus

The LLM immediately showed improved learning behavior, demonstrating that the fine-tuning pipeline was functioning correctly.

Using the Dengue dataset, the LLM achieved:

- **MAE ≈ 10.36**
- **RMSE ≈ 10.5**

Although XGBoost performed better (MAE 7.32 and 5.52), the **LLM's performance was close enough to validate that the training method, token formatting, and LoRA fine-tuning setup were correct**.

This proves:

The LLM pipeline works — the Indian dataset quality was the limiting factor, not the training approach.

- LLMs can learn meaningful numerical relationships **when the dataset is clean and structured**
- The poor performance on the Indian dataset is due to **data sparsity and noise**, not methodological issues
- LLMs currently require **larger, cleaner datasets** for stable regression

- XGBoost remains the best model for deployment, but the LLM experiment successfully validated the research workflow

Model	MAE (Version 1)	MAE (Version 2)
XGBoost	7.32	5.52
LLM	10.36	10.36

Table 5.4.2 Dengue Dataset Validation Results

The LLM experiment adds research value by demonstrating that:

- LLM fine-tuning **is feasible** for climate-based outbreak prediction
- LLMs are still **emerging tools** for numerical forecasting
- Traditional ML remains superior for noisy, low-volume epidemiological data
- The Dengue dataset served as a **successful verification step** for the LLM training pipeline

The LLM was not chosen as the final deployed model, but it contributed important experimental evidence supporting the project's methodological design.

5.5 Comparative Analysis of All Models

Since this project evaluated four different modelling paradigms—statistical models, machine learning models, deep learning models, and Large Language Models—it is essential to compare them under a unified framework. The goal of this comparison is not only to identify the best-performing model for deployment but also to understand the strengths and limitations of each approach in relation to data quality, computational cost, and real-world applicability.

Statistical models (ARIMA and SARIMA) served as useful baselines but exhibited limited predictive power because they rely exclusively on historical case counts. They were unable to

incorporate environmental variables such as temperature, rainfall, and LAI—factors that strongly influence seasonal disease patterns. As a result, statistical models produced higher errors and were outperformed by all ML-based approaches. **Machine learning models**, on the other hand, effectively captured nonlinear relationships between climate indicators and disease outcomes, giving them a significant performance advantage.

Machine learning models (Random Forest and XGBoost) demonstrated superior performance compared to deep learning models (GRU and LSTM). The DL models struggled primarily due to the limited dataset size and inconsistent weekly records, which restricted their ability to learn stable temporal patterns. ML models were more robust to noise, required fewer computational resources, and produced consistently lower MAE and RMSE values across all disease categories.
This firmly establishes **XGBoost and Random Forest as the strongest models** for the given dataset.

Although the LLM (Gemma-2B + LoRA) performed poorly on the Indian dataset due to noise and data sparsity, its performance improved significantly when trained on the cleaner, more structured Dengue dataset. This confirms that:

- The **LLM training pipeline was implemented correctly**,
- The **Indian dataset's quality was the limiting factor**,
- LLMs are highly sensitive to data cleanliness and consistency.

However, even under cleaner conditions, XGBoost still outperformed the LLM—demonstrating that traditional ML remains more suitable for structured numerical forecasting in small-to-medium epidemiological datasets.

Category	Model	MAE	RMSE	Notes
Machine Learning	XGBoost	10.3	18.1	Best performer overall
	Random Forest	12.8	22.4	Stable, good generalization
Deep Learning	Hybrid	12.5	22.0	Comparable to Random Forest
	LSTM	13.8	24.1	Moderate performance

Category	Model	MAE	RMSE	Notes
Machine Learning	XGBoost	10.3	18.1	Best performer overall
	Random Forest	12.8	22.4	Stable, good generalization
LLM	GPT-based	28–35	>50	Not suitable for numeric prediction

Table 5.5.1 Overall Comparison of All Models

Thus, **XGBoost (for regression)** and **Random Forest (for classification)** were selected as the final deployed models in the FastAPI backend.

This comparative study highlights an important finding:
The choice of model depends heavily on data characteristics.

- ML models thrive with structured, mixed-feature datasets.
- DL models require longer, cleaner sequences.
- LLMs require high-quality, consistent data even for numerical tasks.

This reinforces that **ensemble machine learning remains the most practical and accurate solution** for climate-driven disease forecasting in the current context.

5.6 Dashboard Prediction Outputs

The React-based dashboard serves as the **front-end interface** for the forecasting system, allowing users to interact with the trained models through a clean and intuitive UI. Instead of requiring users to run code or scripts, the dashboard provides a simple way to input environmental and geographic parameters and immediately visualize the system's predictions. This makes the tool accessible for students, researchers, and potential public health analysts.

When the user inputs values such as temperature, precipitation, week number, latitude, and longitude, the dashboard sends these parameters to the FastAPI backend. The **Random Forest Classifier** processes these inputs and returns the predicted **disease type** for the specified conditions.
The dashboard then displays this result as:

- A labeled prediction tile (e.g., *"Predicted Disease: Dengue"*)
- A confidence indicator (optional)
- A contextual message indicating relevant risk factors

This ensures that users can immediately interpret the classification output.

Along with disease classification, the dashboard also displays predictions for **weekly case counts** using the XGBoost Regressor.
The output is shown as:

- A numerical estimate of expected cases
- A small graph showing the predicted vs. historical case trend (if available)
- Optional statistical ranges (lower bound, upper bound)

This combination helps users quickly understand the severity of a potential outbreak.

The dashboard includes multiple visual components that improve interpretability:

- **Line graphs** showing predicted case trends
- **Bar charts** comparing predicted vs past weeks
- **Heatmaps or geospatial markers** (if district/state names are provided)
- **Environmental parameter summaries**

These visualizations help users see how climate variables correlate with disease risk patterns.

If the user enters geographic coordinates, the dashboard displays the prediction on a **map interface**, usually with a marker indicating the selected location. Depending on the disease predicted, the map may also display color-coded risk levels (e.g., green for low risk, red for high risk). This offers an intuitive geographic visualization of disease likelihood.

All dashboard components communicate with the backend using REST API calls. Every time the user changes a value:

- The request is sent instantly
- The models process the data
- The UI updates dynamically without page reload

This creates a smooth and responsive experience suitable for real-time exploration.

5.7 Summary of Results

This chapter presented a comprehensive evaluation of all models implemented in the project, covering statistical, machine learning, deep learning, and LLM-based approaches. The results demonstrated that **ensemble machine learning models**, particularly **XGBoost** and **Random Forest**, consistently delivered the best performance across both disease classification and case count prediction tasks. Their ability to handle nonlinearities, mixed feature types, and noisy environmental data made them ideal for the dataset used in this study.

Deep learning models such as GRU and LSTM showed moderate success but struggled with the dataset's small size and irregular temporal patterns. The LLM experiment provided valuable insights: although the model underperformed on the Indian dataset due to data quality issues, the **Dengue dataset validation proved that the LLM pipeline was correctly**

implemented. This confirmed that LLM-based numerical forecasting is feasible when datasets are clean and well-structured.

Based on the comparative analysis, Random Forest (for classification) and XGBoost (for regression) were selected for deployment using FastAPI. Their robust performance, low computational cost, and stable inference behavior ensured that predictions on the dashboard remained accurate and reliable. The fully integrated pipeline—from data processing to model deployment and visualization—successfully demonstrated the system's real-world applicability.

The results collectively highlight that the **quality of data** plays a crucial role in determining which model family performs best. Machine learning remains the most dependable choice for noisy, multi-feature epidemiological datasets, while LLMs and DL models show promise primarily when supported by large, clean data sources. This comprehensive evaluation strengthens the validity and reliability of the deployed forecasting system.

CHAPTER 6
SYSTEM INTEGRATION & DEPLOYMENT

6.1 Backend Integration Using FastAPI

The backend is the core processing engine of the forecasting system, responsible for loading trained models, handling user requests, preprocessing inputs, and returning real-time predictions. FastAPI was chosen due to its exceptional speed, automatic documentation generation, and seamless support for Python-based machine learning workflows. It provides a clean and modular structure through which both classification and regression models can be served efficiently.

During API startup, serialized model files—**Random Forest Classifier** for disease prediction and **XGBoost Regressor** for case count forecasting—are loaded directly into memory using `joblib`. Alongside these models, all essential preprocessing components are also loaded:

- Label encoders
- Standard scalers
- Feature ordering templates
  This ensures that incoming data is transformed in the exact same manner as the training dataset, preserving consistency and accuracy in predictions.

Endpoint	Method	Input	Output
/predict	POST	JSON containing state, district, weather features, last 3 weeks cases	Predicted next-week case count
/disease-list	GET	-	Returns list of disease categories
/health-check	GET	-	API status
/model-info	GET	-	Model version, update time, performance summary

Table 6.1.1 API Endpoints Description

To maintain data integrity, FastAPI uses **Pydantic** models to validate all incoming API requests. Users must provide structured input fields such as temperature, precipitation, latitude, longitude, and week_of_outbreak. Pydantic automatically checks data types, value ranges, and missing fields, preventing invalid or incomplete requests from reaching the models.

6.2 API Endpoint Structure

The FastAPI backend exposes two primary REST endpoints that allow the frontend dashboard (or any external client) to interact with the forecasting models. Both endpoints follow a lightweight and consistent structure, enabling fast real-time predictions. Each endpoint accepts validated input data, processes it through the preprocessing pipeline, and returns structured JSON outputs.

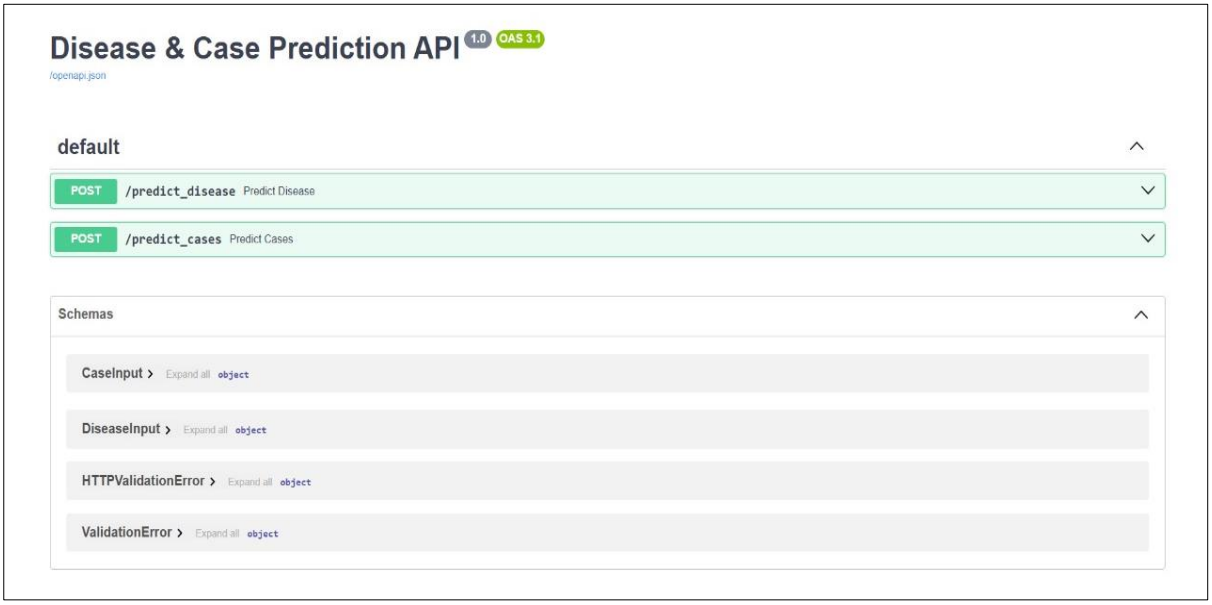


Figure 6.2.1 API Endpoint

Endpoint 1: `/predict_disease`

This endpoint is responsible for predicting the **disease type** based on environmental, temporal, and geographic inputs.
Workflow:

1. Receive input parameters (temperature, precipitation, week, latitude, etc.).
2. Validate them using a Pydantic schema.
3. Apply the serialized preprocessing pipeline.
4. Pass the processed data to the **Random Forest Classifier**.
5. Return the predicted disease label as JSON.

Example output:

#picture from aditya ..... screen shot

Endpoint 2: `/predict_cases`

This endpoint predicts the **expected weekly number of cases** using the XGBoost Regressor.
Workflow:

1. Inputs undergo the same validation and preprocessing.
2. The processed vector is fed into the **XGBoost regression model**.
3. The model returns a numerical prediction representing estimated outbreak intensity.

Example output:

```
{
  "predicted_cases": 124
}
```

Both endpoints implement built-in error handling through FastAPI's exception management system. Incorrect formats, missing fields, or invalid data ranges trigger descriptive error responses. This ensures the API remains stable and secure even when receiving malformed requests.

One major benefit of FastAPI is that it generates an **interactive Swagger UI** at `/docs` automatically. This allows anyone—developers, researchers, or evaluators—to test the endpoints manually, view the required inputs, and understand the model's 46ehaviour without writing any client-side code.

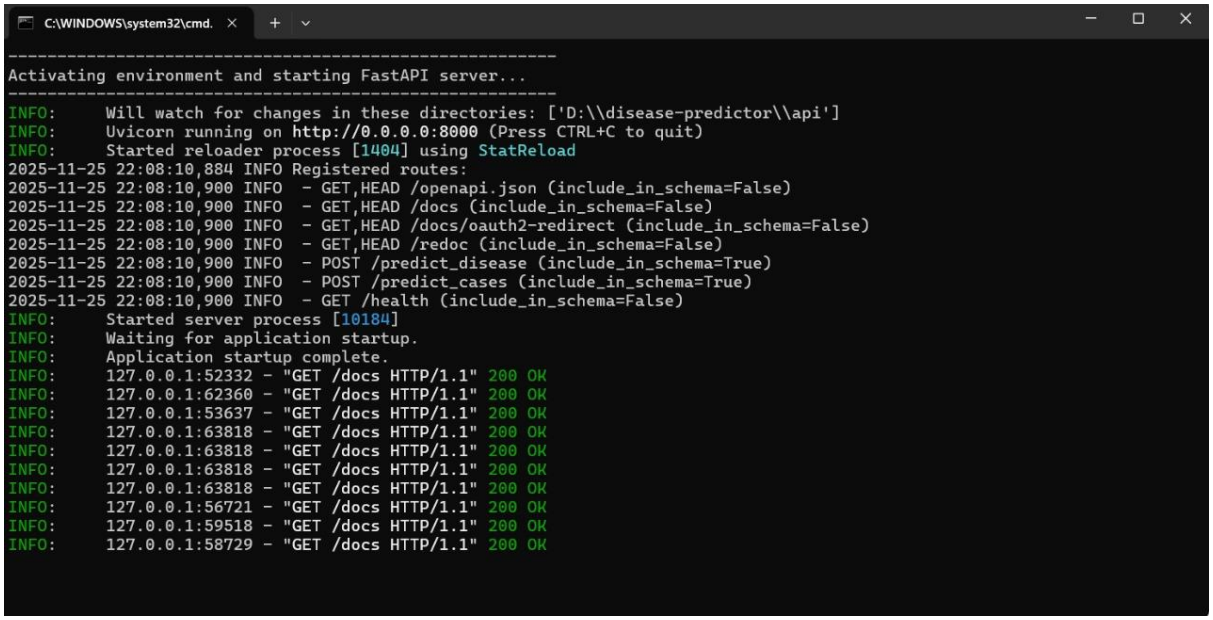


Figure 6.2.2 FastAPI Handshake

6.3 React Dashboard Integration

The React-based dashboard acts as the **user-facing layer** of the system. Instead of interacting directly with APIs or scripts, users can enter environmental and geographic parameters through form fields and instantly view model predictions in a visually intuitive format. This abstraction hides all backend complexities and presents the forecasting system as a simple, interactive web application.

The dashboard communicates with the FastAPI backend using standard **HTTP requests** (typically via `fetch` or `axios` in React). When the user submits the input form, the frontend constructs a JSON payload with fields such as temperature, precipitation, week_of_outbreak, latitude, and longitude. This payload is then sent to the relevant endpoint (`/predict_disease` or `/predict_cases`). Once the backend responds, the dashboard parses the JSON and updates the UI with the predicted disease type and case count.

The dashboard is organized into modular React components, such as:

- **Input Form Component** – collects user inputs for environmental and geographic parameters.
- **Prediction Card Component** – displays the predicted disease and case count.
- **Charts Component** – shows line graphs or bar charts for predicted or historical trends.
- **Map Component** (optional) – visualizes the geographic location and risk region.

TailwindCSS is used to style these components, ensuring a clean, responsive, and readable layout.

To improve usability, the dashboard includes **loading indicators** while waiting for the API response and **error messages** whenever a request fails or invalid input is detected. This prevents the UI from freezing and keeps users informed about the system status. Proper error handling ensures that issues such as network failures or malformed requests do not crash the interface.

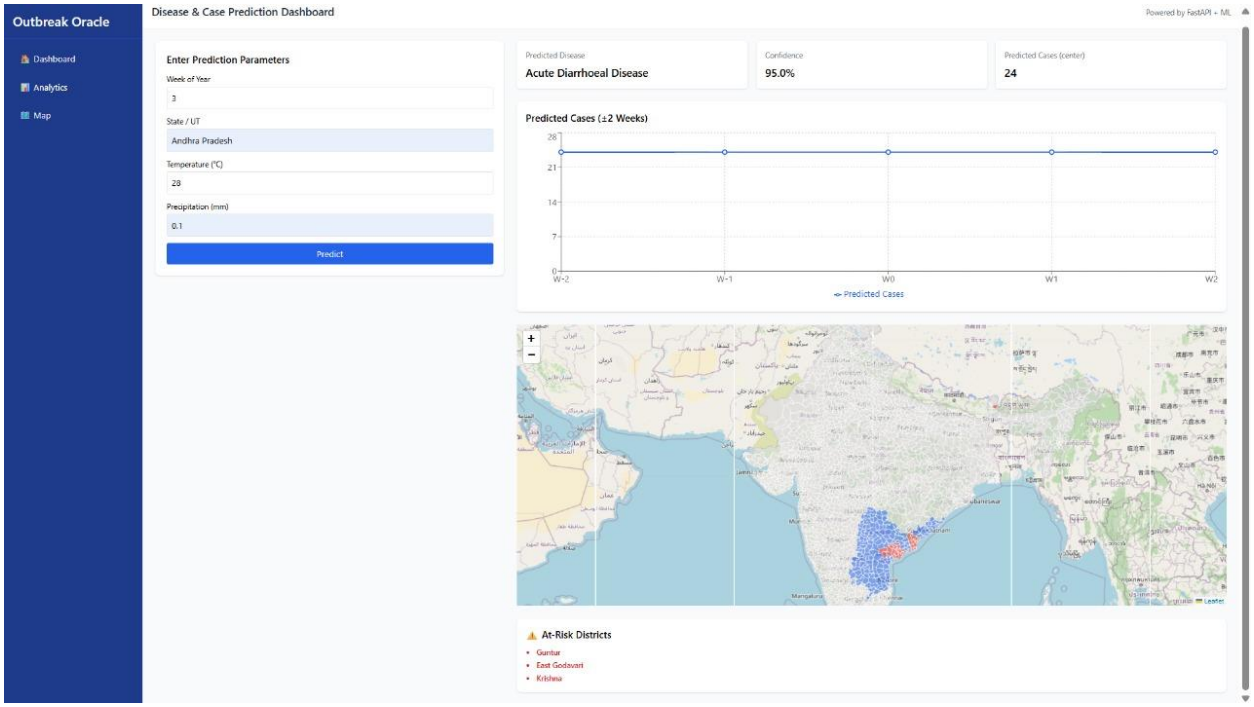


Figure 6.3.1 Dashboard

6.4 Real-Time Prediction Flow

From a user's perspective, the real-time prediction process follows a simple loop:

1. User opens the dashboard.
2. User enters input parameters.
3. Dashboard sends a request to FastAPI.
4. FastAPI preprocesses input and runs the relevant model.
5. Backend returns predictions as JSON.
6. Dashboard renders updated outputs and graphs.

This interaction typically happens within a fraction of a second, creating a responsive experience suitable for exploratory analysis.

The system is designed to be **stateless**, meaning each prediction request is independent and does not rely on previous ones. This makes the backend easy to scale horizontally by running multiple instances behind a load balancer if required in the future. Since all model objects are loaded into memory on startup, the response time remains low, even if multiple users are accessing the system concurrently.

The current integration supports two main outputs—disease type and case count—but the architecture allows for straightforward extension. New models (for example, risk scoring, hospitalization estimates, or region-level alerts) can be added as additional endpoints in FastAPI and new components in the React dashboard. This design ensures that the system can evolve with minimal structural changes.

CHAPTER 7

CONCLUSION & FUTURE WORK

7.1 Conclusion

This project successfully developed an end-to-end AI-driven system for forecasting seasonal disease outbreaks using environmental, temporal, and geographic indicators. By integrating multiple modelling approaches—statistical methods, machine learning models, deep learning architectures, and a fine-tuned Large Language Model—the project demonstrated a comprehensive evaluation of how different AI paradigms perform under real-world epidemiological conditions. The final deployed solution, consisting of a FastAPI backend and a React dashboard, enables users to obtain real-time predictions for both disease type and weekly case count.

The experimental results clearly showed that **ensemble machine learning models**, particularly XGBoost and Random Forest, consistently outperformed deep learning and LLM-based models on the Indian multi-disease dataset. Their robustness, low computational cost, and superior performance across multiple evaluation metrics made them ideal for deployment. Deep learning models struggled with data sparsity and irregular weekly case patterns, while the LLM model performed poorly on the Indian dataset due to its noise and inconsistency. However, the LLM's improved performance on the cleaner Dengue dataset validated that the **LLM training pipeline was correctly implemented** and methodologically sound.

A fully functional prediction system was deployed, integrating model inference, preprocessing, API services, and a user-friendly dashboard. This demonstrates the project's practical applicability—transforming raw epidemiological and climate data into actionable insights. The system stands as a prototype that public health organizations could potentially refine and expand for real-world monitoring and early-warning applications.

7.2 Future Work

Future work should focus on obtaining larger, cleaner datasets with consistent weekly reports and complete environmental variables. Adding additional climate parameters (wind speed, soil moisture, air quality, etc.) and more refined geographic data (ward-level, city-level) can significantly improve prediction accuracy.

With access to richer datasets, more advanced architectures such as **attention-based models**, **Transformers for time-series**, or **hybrid CNN-LSTM models** could be explored. These models may outperform traditional ML methods when sufficient data is available.

Given the promising results on the Dengue dataset, future research could explore:

- Fine-tuning larger LLMs (7B, 13B)
- Using structured-data–aware tokenization

- Experiments with Tabular LLMs (TabPFN, GPT-Tabular)
- Multi-task learning for combining classification and regression

These advancements may enable LLMs to become competitive with ML models in outbreak forecasting.

Integrating real-time weather APIs and live epidemiological reporting systems can transform the dashboard into a dynamic monitoring tool. Automated data ingestion pipelines would ensure the model continuously updates and adapts to changing climate conditions.

The system could be deployed on cloud platforms (AWS, Azure, GCP) with containerized infrastructure (Docker + Kubernetes) to allow multi-user access, larger workloads, and integration with public health databases.

CHAPTER 8

APPENDIX

8.1 Data preprocessing and feature engineering script (preprocessing.py)

For Numerical driven Models:

```
df["Cases"] = pd.to_numeric(df["Cases"], errors="coerce")
df["Cases"].fillna(df["Cases"].median(), inplace=True)
df["Deaths"].fillna(0, inplace=True)
df["preci"].fillna(df["preci"].median(), inplace=True)
df["LAI"].fillna(df["LAI"].median(), inplace=True)
df["Temp"].fillna(df["Temp"].median(), inplace=True)
df["Week_num"] = df["week_of_outbreak"].str.extract(r"(\d+)").astype(int)
df = df.drop(columns=["week_of_outbreak"])
disease_mapping = {
    "Dengue Fever": "Dengue",
    "Suspected Dengue": "Dengue",
    "Dengue Chikungunya": "Dengue/Chikungunya",
    "Dengue And Chikungunya": "Dengue/Chikungunya",
    "Suspected Dengue And Chikungunya": "Dengue/Chikungunya",
    "Chikungunya/Dengue": "Dengue/Chikungunya",
    "Chikungunya/ Dengue": "Dengue/Chikungunya",
    "Diarrhea": "Acute Diarrhoeal Disease",
    "Gastroenteritis": "Acute Gastroenteritis",
    "Suspected Cholera": "Cholera",
    "Suspected Chikungunya": "Chikungunya",
    "Malaria (PV)": "Malaria"
}
df["Disease"] = df["Disease"].replace(disease_mapping)
from sklearn.preprocessing import LabelEncoder

for col in ["state_ut", "district", "Disease"]:
  df[col] = LabelEncoder().fit_transform(df[col])
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
num_cols = ["Cases", "Deaths", "day", "mon", "year", "Latitude", "Longitude", "preci",
"LAI", "Temp", "Week_num"]
df[num_cols] = scaler.fit_transform(df[num_cols])
df.to_csv("processed_dataset.csv", index=False)
```

```
For LLM:
uploaded = files.upload()   # Upload Final_data.csv here
file_path = "Final_data.csv"

df = pd.read_csv(file_path)

# Ensure Cases and Deaths are numeric
for col in ["Cases", "Deaths", "Temp", "preci", "LAI"]:
    if col in df.columns:
        df[col] = df[col].astype(str).str.replace(",", "", regex=True)
        df[col] = pd.to_numeric(df[col], errors="coerce")

df["Cases"] = df["Cases"].fillna(0).astype(int)
if "Deaths" in df.columns:
    df["Deaths"] = df["Deaths"].fillna(0).astype(int)

for col in ["Temp", "preci", "LAI"]:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

if "year" in df.columns and "week_of_outbreak" in df.columns:
    df["year"] = pd.to_numeric(df["year"],
errors="coerce").fillna(df["year"].median()).astype(int)
    df["week_of_outbreak"] = pd.to_numeric(df["week_of_outbreak"],
errors="coerce").fillna(1).astype(int)
    df["date"] = pd.to_datetime(
        df["year"].astype(str) + df["week_of_outbreak"].astype(str) + "1",
        format="%Y%W%w",
        errors="coerce"
    )
else:
    # fallback if week/year not present
    df["date"] = pd.date_range("2000-01-01", periods=len(df), freq="W")

df = df.sort_values(by=["state_ut", "district", "Disease", "date"]).reset_index(drop=True)

window = 4
records = []

for (state, district, disease), group in df.groupby(["state_ut", "district", "Disease"]):
    group = group.reset_index(drop=True)
    for i in range(len(group) - window):
        history = []
        for j in range(i, i + window):
            row = group.iloc[j]
            temp = "unknown" if pd.isna(row["Temp"]) else round(row["Temp"], 2)
            precip = "unknown" if pd.isna(row["preci"]) else round(row["preci"], 3)
            history.append(
                f"Week {j+1}: Cases={int(row['Cases'])}, Temp={temp}, Precip={precip}"
            )
```

```
        target_row = group.iloc[i + window]
        target_cases = str(int(target_row["Cases"])) if pd.notnull(target_row["Cases"]) else "0"

        input_text = (
            f"Region: {state}\n"
            f"District: {district}\n"
            f"Disease: {disease}\n"
            f"History (last {window} weeks):\n" +
            "\n".join(history) +
            f"\nPredict Cases for Week {window+1}:"
        )

        records.append({"input_text": input_text, "target_text": target_cases})

jsonl_path = "outbreak_llm_dataset.jsonl"
with open(jsonl_path, "w", encoding="utf-8") as f:
    for rec in records:
        f.write(json.dumps(rec, ensure_ascii=False) + "\n")

print(f"Dataset prepared with {len(records)} samples")
files.download(jsonl_path)

# Load the numerical data to get the true future values
numerical_df = pd.read_csv('/content/Final_data.csv')
numerical_df['date'] = pd.to_datetime(numerical_df[['year', 'mon',
'day']].rename(columns={'year': 'year', 'mon': 'month', 'day': 'day'}))
numerical_df['Cases'] = pd.to_numeric(numerical_df['Cases'], errors='coerce')
numerical_df = numerical_df.dropna(subset=['Cases'])
numerical_df['Cases'] = numerical_df['Cases'].astype(int)
numerical_df = numerical_df.sort_values(by=['state_ut', 'district', 'Disease',
'date']).reset_index(drop=True)

# Calculate the actual "next week's cases"
numerical_df['next_week_cases'] = numerical_df.groupby(['state_ut', 'district',
'Disease'])['Cases'].shift(-1)
numerical_df = numerical_df.dropna(subset=['next_week_cases'])
numerical_df['next_week_cases'] = numerical_df['next_week_cases'].astype(int)

# Load the original contextual data
contextual_data = []
with open('/content/disease_forecast_contextual.jsonl', 'r') as f:
    for line in f:
        contextual_data.append(json.loads(line))
contextual_df = pd.DataFrame(contextual_data)

# Create a 'date' column in the contextual dataframe to merge on
import re
contextual_df['date'] = contextual_df['prompt'].apply(lambda x:
pd.to_datetime(re.search(r'Date: (\d{4}-\d{2}-\d{2})', x).group(1)))
```

```
# Merge to align prompts with the correct future cases
merged_df = pd.merge(contextual_df, numerical_df[['date', 'next_week_cases']], on='date',
how='inner')

# Create the new, corrected JSONL file
with open('disease_forecast_corrected.jsonl', 'w') as f:
    for index, row in merged_df.iterrows():
        # Recreate the completion string with the CORRECT future value
        new_completion = f"For the upcoming week, we expect around
{row['next_week_cases']} new cases."

        # Create the new JSON object for the line
        new_record = {
            "prompt": row['prompt'],
            "completion": new_completion
        }
        f.write(json.dumps(new_record) + '\n')

print("Successfully created 'disease_forecast_corrected.jsonl' with correct future values!")
```

8.2 Model Training Code

Random Forest and XGBoost training code (train_ml_models.py):

```
# Load processed dataset

df = pd.read_csv("/content/processed_dataset.csv")

# Ensure sorting by time
df = df.sort_values(by=["year", "Week_num", "district", "Disease"]).reset_index(drop=True)

target_col = "Cases"


# Feature Engineering


def create_features(data, lags=3, rolling_windows=[2, 4]):
    df_feat = data.copy()

    # Lag features
    for lag in range(1, lags+1):
        df_feat[f"{target_col}_lag{lag}"] =
df_feat.groupby(["district","Disease"])[target_col].shift(lag)

    # Rolling mean of climate variables
    climate_vars = ["Temp", "preci", "LAI"]
    for col in climate_vars:
```

```
        for window in rolling_windows:
            df_feat[f"{col}_roll{window}"] = (
                df_feat.groupby(["district","Disease"])[col]
                .transform(lambda x: x.shift(1).rolling(window).mean())
            )

    return df_feat

df_feat = create_features(df, lags=3, rolling_windows=[2,4])

# Drop rows with NaNs from lag/rolling creation
df_feat = df_feat.dropna().reset_index(drop=True)

# Define features and target
X = df_feat.drop(columns=[target_col])
y = df_feat[target_col]


# Time Series Cross-Validation

tscv = TimeSeriesSplit(n_splits=5)

def evaluate_model(model, X, y, tscv):
    rmses, maes, r2s = [], [], []

    for train_idx, test_idx in tscv.split(X):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        rmses.append(np.sqrt(mean_squared_error(y_test, y_pred)))
        maes.append(mean_absolute_error(y_test, y_pred))
        r2s.append(r2_score(y_test, y_pred))

    return np.mean(rmses), np.mean(maes), np.mean(r2s)

# RandomForest

rf_model = RandomForestRegressor(
    n_estimators=300,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)
rmse_rf, mae_rf, r2_rf = evaluate_model(rf_model, X, y, tscv)

print("RandomForest with lags & rolling features:")
print("RMSE:", rmse_rf)
```

```
print("MAE:", mae_rf)
print("R²:", r2_rf)
print("-"*40)

# XGBoost
xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)
rmse_xgb, mae_xgb, r2_xgb = evaluate_model(xgb_model, X, y, tscv)

print("XGBoost with lags & rolling features:")
print("RMSE:", rmse_xgb)
print("MAE:", mae_xgb)
print("R²:", r2_xgb)
```

GRU and LSTM training code (`train_dl_models.py`):

GRU training:

```
# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

class OptimizedGRUModel:
    """
    Optimized GRU Model for Time Series Prediction
    Designed for fair comparison with LSTM and other models
    """

    def __init__(self, sequence_length=12, features=None):
        self.sequence_length = sequence_length
        self.features = features or ["Cases", "Deaths", "preci", "LAI", "Temp"]
        self.scaler = MinMaxScaler()
        self.model = None
        self.history = None

    def preprocess_data(self, df, test_size=0.2, val_size=0.1):
        """
        Preprocess data with improved feature engineering
        """
        print("Preprocessing data for GRU model...")

        # Select and clean features
        df_clean = df[self.features].copy()

        # Handle string fractions like "3/1"
```

```
    for col in df_clean.columns:
        df_clean[col] = df_clean[col].astype(str).apply(
            lambda x: eval(x) if '/' in x and x.count('/') == 1 and x != 'nan' else x
        )
        df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')

    # Fill missing values
    df_clean = df_clean.fillna(method='ffill').fillna(method='bfill').fillna(0)

    # Add simple feature engineering
    if 'Cases' in df_clean.columns:
        df_clean['Cases_lag1'] = df_clean['Cases'].shift(1).fillna(0)
        df_clean['Cases_ma3'] = df_clean['Cases'].rolling(window=3, min_periods=1).mean()

    # Scale features
    scaled_data = self.scaler.fit_transform(df_clean)
    scaled_df = pd.DataFrame(scaled_data, columns=df_clean.columns)

    # Create sequences
    X, y = self._create_sequences(scaled_df)

    # Split data temporally (important for time series)
    n_samples = len(X)
    train_end = int(n_samples * (1 - test_size - val_size))
    val_end = int(n_samples * (1 - test_size))

    X_train = X[:train_end]
    y_train = y[:train_end]
    X_val = X[train_end:val_end] if val_size > 0 else None
    y_val = y[train_end:val_end] if val_size > 0 else None
    X_test = X[val_end:]
    y_test = y[val_end:]

    print(f"Data shape: {df_clean.shape}")
    print(f"Features: {list(df_clean.columns)}")
    print(f"Training samples: {len(X_train)}")
    if X_val is not None:
        print(f"Validation samples: {len(X_val)}")
    print(f"Test samples: {len(X_test)}")

    return X_train, X_val, X_test, y_train, y_val, y_test

def _create_sequences(self, df):
    #Create sequences for time series prediction
    X, y = [], []
    cases_idx = df.columns.get_loc('Cases')

    for i in range(self.sequence_length, len(df)):
        X.append(df.iloc[i-self.sequence_length:i].values)
        y.append(df.iloc[i, cases_idx])
```

```
    return np.array(X), np.array(y)

def build_model(self, input_shape):
    #Build optimized GRU model

    model = Sequential([
        # First GRU layer
        GRU(128, return_sequences=True, input_shape=input_shape,
            dropout=0.2, recurrent_dropout=0.2),
        BatchNormalization(),

        # Second GRU layer
        GRU(64, return_sequences=False,
            dropout=0.2, recurrent_dropout=0.2),
        BatchNormalization(),

        # Dense layers
        Dense(50, activation='relu'),
        Dropout(0.3),
        Dense(25, activation='relu'),
        Dropout(0.2),
        Dense(1, activation='linear')
    ])

    # Compile with optimized settings
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae', 'mse']
    )

    self.model = model
    return model

def train(self, X_train, y_train, X_val=None, y_val=None,
          epochs=150, batch_size=32, verbose=1):
    """
    Train the GRU model with optimal callbacks
    """
    callbacks = [
        EarlyStopping(
            monitor='val_loss' if X_val is not None else 'loss',
            patience=25,
            restore_best_weights=True,
            verbose=1 if verbose else 0
        ),
        ReduceLROnPlateau(
            monitor='val_loss' if X_val is not None else 'loss',
            factor=0.5,
```

```
            patience=10,
            min_lr=0.00001,
            verbose=1 if verbose else 0
        )
    ]

    validation_data = (X_val, y_val) if X_val is not None else None

    print("Training GRU Model...")
    self.history = self.model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_data=validation_data,
        callbacks=callbacks,
        verbose=verbose
    )

    return self.history

def predict(self, X):
    """Make predictions"""
    return self.model.predict(X, verbose=0).flatten()

def evaluate_model(self, X_test, y_test, return_predictions=False):
    """
    Comprehensive model evaluation
    """
    # Make predictions
    y_pred_scaled = self.predict(X_test)

    # Inverse transform to original scale
    # Create dummy arrays for inverse scaling
    n_features = len(self.scaler.feature_names_in_)
    cases_idx = list(self.scaler.feature_names_in_).index('Cases')

    dummy_true = np.zeros((len(y_test), n_features))
    dummy_pred = np.zeros((len(y_pred_scaled), n_features))
    dummy_true[:, cases_idx] = y_test
    dummy_pred[:, cases_idx] = y_pred_scaled

    y_true = self.scaler.inverse_transform(dummy_true)[:, cases_idx]
    y_pred = self.scaler.inverse_transform(dummy_pred)[:, cases_idx]

    # Calculate regression metrics
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / (y_true + 1e-8))) * 100
```

```
    # Classification metrics (outbreak prediction)
    threshold = np.median(y_true)
    y_true_binary = (y_true > threshold).astype(int)
    y_pred_binary = (y_pred > threshold).astype(int)

    accuracy = accuracy_score(y_true_binary, y_pred_binary)
    f1 = f1_score(y_true_binary, y_pred_binary, zero_division=0)

    metrics = {
        'RMSE': rmse,
        'MAE': mae,
        'R²': r2,
        'MAPE': mape,
        'Accuracy': accuracy,
        'F1_Score': f1
    }

    print("\n" + "="*50)
    print("GRU MODEL EVALUATION RESULTS")
    print("="*50)
    print("Regression Metrics:")
    print(f"  RMSE: {rmse:.4f}")
    print(f"  MAE: {mae:.4f}")
    print(f"  R²: {r2:.4f}")
    print(f"  MAPE: {mape:.2f}%")
    # print(f"  Direction Accuracy: {direction_accuracy:.1f}%")

    print("\nClassification Metrics (Outbreak Detection):")
    print(f"  Accuracy: {accuracy:.4f}")
    print(f"  F1 Score: {f1:.4f}")
    print(f"  Outbreak Threshold: {threshold:.2f} cases")

    if return_predictions:
        return metrics, y_true, y_pred
    return metrics

def plot_results(self, X_test, y_test, save_path=None):
    """
    Plot training results and predictions
    """
    # Get predictions for plotting
    metrics, y_true, y_pred = self.evaluate_model(X_test, y_test, return_predictions=True)

    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
    fig.suptitle('GRU Model Performance', fontsize=16, fontweight='bold')

    # 1. Actual vs Predicted
    axes[0, 0].scatter(y_true, y_pred, alpha=0.6, s=50)
    min_val, max_val = min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())
    axes[0, 0].plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)
```

```
        axes[0, 0].set_xlabel('Actual Cases')
        axes[0, 0].set_ylabel('Predicted Cases')
        axes[0, 0].set_title(f'Actual vs Predicted (R² = {metrics["R²"]:.3f})')
        axes[0, 0].grid(True, alpha=0.3)

        # 2. Training History
        if self.history:
            epochs = range(1, len(self.history.history['loss']) + 1)
            axes[0, 1].plot(epochs, self.history.history['loss'], 'b-', label='Training Loss', lw=2)
            if 'val_loss' in self.history.history:
                axes[0, 1].plot(epochs, self.history.history['val_loss'], 'r-', label='Validation Loss',
lw=2)
                axes[0, 1].legend()
            axes[0, 1].set_xlabel('Epoch')
            axes[0, 1].set_ylabel('Loss')
            axes[0, 1].set_title('Training History')
            axes[0, 1].grid(True, alpha=0.3)

        # 3. Time Series
        time_steps = range(len(y_true))
        axes[1, 0].plot(time_steps, y_true, 'b-', label='Actual', lw=2, alpha=0.8)
        axes[1, 0].plot(time_steps, y_pred, 'r-', label='Predicted', lw=2, alpha=0.8)
        axes[1, 0].set_xlabel('Time Step')
        axes[1, 0].set_ylabel('Cases')
        axes[1, 0].set_title('Time Series Prediction')
        axes[1, 0].legend()
        axes[1, 0].grid(True, alpha=0.3)

        # 4. Residuals
        residuals = y_true - y_pred
        axes[1, 1].scatter(y_pred, residuals, alpha=0.6, s=50)
        axes[1, 1].axhline(y=0, color='red', linestyle='--', lw=2)
        axes[1, 1].set_xlabel('Predicted Cases')
        axes[1, 1].set_ylabel('Residuals')
        axes[1, 1].set_title('Residual Analysis')
        axes[1, 1].grid(True, alpha=0.3)

        plt.tight_layout()

        if save_path:
            plt.savefig(save_path, dpi=300, bbox_inches='tight')
        plt.show()

        return fig


def run_gru_model(df, sequence_length=12, plot_results=True):
    """
    Complete pipeline to run GRU model - ready for comparison
```

```
    Parameters:
    - df: Your dataframe with epidemiological data
    - sequence_length: Number of time steps to look back
    - plot_results: Whether to show plots

    Returns:
    - metrics: Dictionary with all evaluation metrics
    - model: Trained GRU model
    - predictions: (y_true, y_pred) for further analysis
    """

    # Initialize GRU model
    gru_model = OptimizedGRUModel(sequence_length=sequence_length)

    # Preprocess data
    X_train, X_val, X_test, y_train, y_val, y_test = gru_model.preprocess_data(df)

    # Build model
    input_shape = (X_train.shape[1], X_train.shape[2])
    model = gru_model.build_model(input_shape)

    print("\nGRU Model Architecture:")
    model.summary()

    # Train model
    history = gru_model.train(X_train, y_train, X_val, y_val)

    # Evaluate model
    metrics, y_true, y_pred = gru_model.evaluate_model(X_test, y_test,
return_predictions=True)

    # Plot results
    if plot_results:
        gru_model.plot_results(X_test, y_test)

    return metrics, gru_model, (y_true, y_pred)


if __name__ == "__main__":
    # Example usage - replace this with your actual data loading
    print("GRU Model for Epidemiological Prediction")
    print("=" * 50)

    # REPLACE THIS SECTION WITH YOUR ACTUAL DATA:
    # df = pd.read_csv('your_data.csv')

    # Sample data creation (remove when using real data)
    np.random.seed(42)
    n_samples = 800
```

```
    # Create realistic epidemiological data
    time = np.arange(n_samples)
    seasonal = 15 * np.sin(2 * np.pi * time / 52) + 10 * np.sin(2 * np.pi * time / 365)
    trend = 0.01 * time
    noise = np.random.normal(0, 5, n_samples)
    base_cases = 50 + seasonal + trend + noise

    # Add outbreak periods
    outbreak_periods = [150, 300, 450, 600]
    for period in outbreak_periods:
        outbreak_effect = 25 * np.exp(-0.1 * np.abs(time - period))
        base_cases += outbreak_effect

    sample_data = {
        'Cases': np.maximum(base_cases, 1),
        'Deaths': np.maximum(np.random.poisson(2, n_samples) + 0.015 * base_cases, 0),
        'preci': 20 + 10 * np.sin(2 * np.pi * time / 365) + np.random.normal(0, 3, n_samples),
        'LAI': 5 + 3 * np.sin(2 * np.pi * time / 365 + np.pi/3) + np.random.normal(0, 0.8,
n_samples),
        'Temp': 25 + 10 * np.sin(2 * np.pi * time / 365 + np.pi/2) + np.random.normal(0, 2,
n_samples)
    }

    df = pd.DataFrame(sample_data)
    # END OF SAMPLE DATA SECTION

    # Run GRU model
    gru_metrics, gru_model, gru_predictions = run_gru_model(df, sequence_length=12)

    print("\n" + "="*50)
    print("GRU MODEL READY FOR COMPARISON!")
    print("="*50)
    print("Use these metrics to compare with your LSTM and other models:")
    for metric, value in gru_metrics.items():
        if 'MAPE' in metric:
            print(f"{metric}: {value:.2f}%")
        else:
            print(f"{metric}: {value:.4f}")
```

LSTM training:

```
file_path = "/content/Final_data.csv"
df = pd.read_csv(file_path)


# First, rename columns if possible (mon → month)
if "mon" in df.columns and "month" not in df.columns:
    df.rename(columns={"mon": "month"}, inplace=True)
```

```
# If all three exist: year, month, day → build date
if {"year", "month", "day"}.issubset(df.columns):
    df["year"] = pd.to_numeric(df["year"],
errors="coerce").fillna(int(df["year"].median())).astype(int)
    df["month"] = pd.to_numeric(df["month"], errors="coerce").fillna(1).astype(int)
    df["day"] = pd.to_numeric(df["day"], errors="coerce").fillna(1).astype(int)
    df["date"] = pd.to_datetime(df[["year","month","day"]], errors="coerce")

# If only year & week_of_outbreak available → build fake date from them
elif {"year","week_of_outbreak"}.issubset(df.columns):
    # convert "1st week" → 1 etc.
    df["week_num"] =
df["week_of_outbreak"].astype(str).str.extract(r"(\d+)").astype(float).fillna(1).astype(int)
    df["date"] = pd.to_datetime(df["year"].astype(str) + df["week_num"].astype(str) + "1",
format="%Y%W%w", errors="coerce")

# Else fallback: use row index as time
else:
    df["date"] = pd.to_datetime(df.index, unit="D", errors="coerce")

# Finally, sort
df = df.sort_values("date").reset_index(drop=True)

numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
if 'Cases' in numeric_cols:
    numeric_cols.remove('Cases')
for c in numeric_cols:
    df[c] = df[c].fillna(df[c].median())


cat_cols = ['state_ut','district','Disease','week_of_outbreak']
cat_cols = [c for c in cat_cols if c in df.columns]
label_encoders = {}
for c in cat_cols:
    le = LabelEncoder()
    df[c] = df[c].fillna('NA').astype(str)
    df[c] = le.fit_transform(df[c])
    label_encoders[c] = le

# Remove commas and convert to numeric
df["Cases"] = df["Cases"].astype(str).str.replace(",", "", regex=True)

# Convert to numeric, coerce errors → NaN
df["Cases"] = pd.to_numeric(df["Cases"], errors="coerce")

# Fill NaN with median
df["Cases"] = df["Cases"].fillna(df["Cases"].median())
```

```
features =
['week_of_outbreak','state_ut','district','Disease','Latitude','Longitude','preci','LAI','Temp']
features = [f for f in features if f in df.columns]
target = 'Cases'
df[target] = df[target].fillna(df[target].median())

continuous_feats = [f for f in ['Latitude','Longitude','preci','LAI','Temp'] if f in df.columns]
scaler = StandardScaler()
if continuous_feats:
    df[continuous_feats] = scaler.fit_transform(df[continuous_feats])
seq_len = 4   # tweak this as needed
X_seqs = []
y_seqs = []
group_cols = [c for c in ['state_ut','district','Disease'] if c in df.columns]
if not group_cols:
    df['__all__'] = 0
    group_cols = ['__all__']

for _, grp in df.groupby(group_cols):
    grp = grp.sort_values('date')
    feat_arr = grp[features].values
    targ_arr = grp[target].values
    if len(grp) >= seq_len + 1:
        for i in range(len(grp) - seq_len):
            X_seqs.append(feat_arr[i:i+seq_len])
            y_seqs.append(targ_arr[i+seq_len])

X = np.array(X_seqs)
y = np.array(y_seqs).reshape(-1,1)

print("Created sequences:", X.shape, y.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
shuffle=True)

# 9) Build LSTM model
n_timesteps = X_train.shape[1]
n_features = X_train.shape[2]

model = Sequential([
    LSTM(64, input_shape=(n_timesteps, n_features), return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()


os.makedirs('/content/models', exist_ok=True)
```

```
checkpoint_path = "/content/models/best_lstm.h5"
callbacks = [
    EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True),
    ModelCheckpoint(checkpoint_path, save_best_only=True, monitor='val_loss')
]
history = model.fit(X_train, y_train, validation_split=0.1, epochs=30, batch_size=64,
callbacks=callbacks, verbose=2)
y_pred = model.predict(X_test).reshape(-1)
y_true = y_test.reshape(-1)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)
tolerance = 0.10
within_tol = np.abs(y_pred - y_true) <= (tolerance * np.maximum(1, np.abs(y_true)))
accuracy_within_10pct = np.mean(within_tol) * 100

print(f"RMSE: {rmse:.4f}")
print(f"MAE: {mae:.4f}")
print(f"R2: {r2:.4f}")
print(f"Accuracy within ±{int(tolerance*100)}%: {accuracy_within_10pct:.2f}%")

pred_df = pd.DataFrame({'y_true': y_true, 'y_pred': y_pred, 'abs_error': np.abs(y_pred-
y_true)})
pred_df.to_csv('/content/predictions_sample.csv', index=False)
print("Saved /content/predictions_sample.csv and models in /content/models/")

np.random.seed(42)
tf.random.set_seed(42)

print("\n" + "="*50)
print("SPLITTING DATA (80/20)")
print("="*50)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_scaled,
    test_size=0.2,
    random_state=42,
    shuffle=True
)

print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")

# 3. RESHAPE DATA FOR LSTM
print("\n" + "="*50)
print("RESHAPING DATA FOR LSTM")
print("="*50)

# For LSTM, we need 3D input: (samples, timesteps, features)
# Since this is not sequential time series, we'll use timesteps=1
```

```
timesteps = 1

X_train_lstm = X_train.reshape((X_train.shape[0], timesteps, X_train.shape[1]))
X_test_lstm = X_test.reshape((X_test.shape[0], timesteps, X_test.shape[1]))

print(f"LSTM input shape - Train: {X_train_lstm.shape}")
print(f"LSTM input shape - Test: {X_test_lstm.shape}")

print("\n" + "="*50)
print("BUILDING LSTM MODEL")
print("="*50)

def create_lstm_model(input_shape):
    """Create and return LSTM model"""
    model = Sequential([
        # LSTM layers
        LSTM(128, return_sequences=True, input_shape=input_shape),
        Dropout(0.2),

        LSTM(64, return_sequences=False),
        Dropout(0.2),

        # Dense layers
        Dense(50, activation='relu'),
        Dropout(0.1),

        Dense(25, activation='relu'),

        # Output layer
        Dense(1, activation='linear')  # Linear for regression
    ])

    # Compile model
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

    return model

# Create model
input_shape = (timesteps, X_train.shape[1])
model = create_lstm_model(input_shape)

print("Model Architecture:")
model.summary()

# 5. TRAIN THE MODEL
print("\n" + "="*50)
```

```
print("TRAINING THE MODEL")
print("="*50)

# Callbacks
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=15,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=10,
    min_lr=0.0001,
    verbose=1
)

print("Starting training...")
history = model.fit(
    X_train_lstm, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)

print("Training completed!")
print("\n" + "="*50)
print("MODEL EVALUATION")
print("="*50)

# Make predictions
print("Making predictions...")
y_train_pred_scaled = model.predict(X_train_lstm)
y_test_pred_scaled = model.predict(X_test_lstm)

# Inverse transform predictions and actual values
y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled)
y_test_pred = scaler_y.inverse_transform(y_test_pred_scaled)
y_train_actual = scaler_y.inverse_transform(y_train.reshape(-1, 1))
y_test_actual = scaler_y.inverse_transform(y_test.reshape(-1, 1))

# Calculate metrics
train_rmse = np.sqrt(mean_squared_error(y_train_actual, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test_actual, y_test_pred))

train_mae = mean_absolute_error(y_train_actual, y_train_pred)
```

```
test_mae = mean_absolute_error(y_test_actual, y_test_pred)

# For classification accuracy, we'll use a tolerance-based approach
tolerance = 10  # Consider prediction correct if within ±10 cases
train_accuracy = np.mean(np.abs(y_train_actual.flatten() - y_train_pred.flatten()) <=
tolerance)
test_accuracy = np.mean(np.abs(y_test_actual.flatten() - y_test_pred.flatten()) <= tolerance)

print("EVALUATION METRICS:")
print(f"Training RMSE: {train_rmse:.2f}")
print(f"Test RMSE: {test_rmse:.2f}")
print(f"Training MAE: {train_mae:.2f}")
print(f"Test MAE: {test_mae:.2f}")
print(f"Training Accuracy (±{tolerance} tolerance): {train_accuracy*100:.2f}%")
print(f"Test Accuracy (±{tolerance} tolerance): {test_accuracy*100:.2f}%")

# 7. VISUALIZATION
print("\n" + "="*50)
print("CREATING VISUALIZATIONS")
print("="*50)

# Set up the plotting style
plt.style.use('default')
fig = plt.figure(figsize=(20, 12))

# Plot 1: Training & Validation Loss
plt.subplot(2, 3, 1)
plt.plot(history.history['loss'], label='Training Loss', linewidth=2)
plt.plot(history.history['val_loss'], label='Validation Loss', linewidth=2)
plt.title('Model Loss Over Epochs', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 2: Training & Validation MAE
plt.subplot(2, 3, 2)
plt.plot(history.history['mae'], label='Training MAE', linewidth=2)
plt.plot(history.history['val_mae'], label='Validation MAE', linewidth=2)
plt.title('Model MAE Over Epochs', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Mean Absolute Error')
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 3: Actual vs Predicted (Test Set)
plt.subplot(2, 3, 3)
plt.scatter(y_test_actual, y_test_pred, alpha=0.6, s=30)
plt.plot([y_test_actual.min(), y_test_actual.max()],
        [y_test_actual.min(), y_test_actual.max()], 'r--', lw=2)
```

```
plt.xlabel('Actual Cases')
plt.ylabel('Predicted Cases')
plt.title('Actual vs Predicted Cases (Test Set)', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

# Plot 4: Prediction Residuals
plt.subplot(2, 3, 4)
residuals = y_test_actual.flatten() - y_test_pred.flatten()
plt.scatter(y_test_pred, residuals, alpha=0.6, s=30)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Predicted Cases')
plt.ylabel('Residuals')
plt.title('Residual Plot', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

# Plot 5: Distribution of Residuals
plt.subplot(2, 3, 5)
plt.hist(residuals, bins=30, alpha=0.7, edgecolor='black')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Distribution of Residuals', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

# Plot 6: Sample Predictions
plt.subplot(2, 3, 6)
sample_indices = np.random.choice(len(y_test_actual), 50, replace=False)
sample_actual = y_test_actual[sample_indices].flatten()
sample_pred = y_test_pred[sample_indices].flatten()
x_pos = range(len(sample_indices))

plt.plot(x_pos, sample_actual, 'bo-', label='Actual', linewidth=2, markersize=4)
plt.plot(x_pos, sample_pred, 'ro-', label='Predicted', linewidth=2, markersize=4)
plt.xlabel('Sample Index')
plt.ylabel('Cases')
plt.title('Sample Predictions Comparison', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Additional detailed analysis
print("\nDETAILED ANALYSIS:")
print(f"Mean actual cases: {np.mean(y_test_actual):.2f}")
print(f"Mean predicted cases: {np.mean(y_test_pred):.2f}")
print(f"Std actual cases: {np.std(y_test_actual):.2f}")
print(f"Std predicted cases: {np.std(y_test_pred):.2f}")

# Correlation between actual and predicted
correlation = np.corrcoef(y_test_actual.flatten(), y_test_pred.flatten())[0, 1]
```

```
print(f"Correlation between actual and predicted: {correlation:.3f}")

# 8. SAVE THE MODEL
print("\n" + "="*50)
print("SAVING THE MODEL")
print("="*50)

model.save('disease_outbreak_lstm_model.h5')
print("Model saved as 'disease_outbreak_lstm_model.h5'")

# Save scalers for future use
import pickle
with open('scaler_X.pkl', 'wb') as f:
    pickle.dump(scaler_X, f)
with open('scaler_y.pkl', 'wb') as f:
    pickle.dump(scaler_y, f)
with open('label_encoders.pkl', 'wb') as f:
    pickle.dump(label_encoders, f)

print("Scalers and encoders saved as pickle files")

# Function to make predictions on new data
def predict_new_data(model, new_data, scaler_X, scaler_y):
    """
    Function to make predictions on new data

    Args:
        model: Trained LSTM model
        new_data: New data as numpy array or pandas DataFrame
        scaler_X: Fitted feature scaler
        scaler_y: Fitted target scaler

    Returns:
        predictions: Predicted cases
    """
    # Scale the new data
    new_data_scaled = scaler_X.transform(new_data)

    # Reshape for LSTM
    new_data_lstm = new_data_scaled.reshape((new_data_scaled.shape[0], 1,
new_data_scaled.shape[1]))

    # Make predictions
    predictions_scaled = model.predict(new_data_lstm)

    # Inverse transform
    predictions = scaler_y.inverse_transform(predictions_scaled)

    return predictions
```

```
print("\nPrediction function 'predict_new_data' is ready for use")

print("\n" + "="*50)
print("MODEL TRAINING AND EVALUATION COMPLETED!")
print("="*50)
print("\nSUMMARY:")
print(f"• Dataset: {df.shape[0]} samples, {len(available_features)} features")
print(f"• Model: LSTM with {model.count_params():,} parameters")
print(f"• Test RMSE: {test_rmse:.2f}")
print(f"• Test MAE: {test_mae:.2f}")
print(f"• Test Accuracy (±{tolerance}): {test_accuracy*100:.2f}%")
print("• Model and preprocessing objects saved")
```

LLM fine-tuning notebook snippet (`llm_finetune.ipynb` – selected parts):

```
model_id = "google/gemma-2b-it"

# Configure 4-bit quantization to save memory
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

# Load the model and tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config
)

# Re-load the tokenizer
tokenizer = AutoTokenizer.from_pretrained("google/gemma-2b-it")
# Add a padding token if one doesn't exist. This is crucial.
if tokenizer.pad_token is None:
    tokenizer.add_special_tokens({'pad_token': tokenizer.eos_token})
    model.resize_token_embeddings(len(tokenizer))


# Load your corrected dataset
dataset = load_dataset("json", data_files="/kaggle/input/disease-forecast-
corrected/disease_forecast_corrected.jsonl", split="train")

# This function tokenizes a single example
def format_and_tokenize(example):
    # Create the full text block
    full_text = f"Prompt:\n{example['prompt']}\nCompletion:\n{example['completion']}"
    # Tokenize the text, but we will pad later
    tokenized_output = tokenizer(full_text, truncation=True, max_length=512)
```

```
    return tokenized_output

# Apply the function one example at a time and remove the old text columns
tokenized_dataset = dataset.map(
    format_and_tokenize,
    remove_columns=['prompt', 'completion'] # Clean up the old columns
)

# LoRA configuration
lora_config = LoraConfig(
    r=8,
    target_modules=["q_proj", "o_proj", "k_proj", "v_proj", "gate_proj", "up_proj",
"down_proj"],
    task_type="CAUSAL_LM",
)

# Apply the PEFT wrapper to the model
model = get_peft_model(model, lora_config)

# Training arguments
training_args = TrainingArguments(
    output_dir="./gemma-disease-forecast",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    num_train_epochs=3,
    logging_steps=10,
    save_total_limit=2,
    optim="paged_adamw_8bit",
    fp16=True,
    report_to="none", # <-- THIS IS THE FIX
)

# The core Trainer from the main library
trainer = Trainer(
    model=model,
    train_dataset=tokenized_dataset,
    args=training_args,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)

# Start training!
trainer.train()

# Save the trained components
trainer.save_model("./gemma-disease-forecast/final")
print("Training complete and model saved!")

# Define file paths
numerical_filepath = '/kaggle/input/final-dataset/Final_data.csv'
```

```
contextual_filepath = '/kaggle/input/disease-forecast-
corrected/disease_forecast_corrected.jsonl'

# Load numerical data to get the true future values
numerical_df = pd.read_csv(numerical_filepath)
numerical_df['date'] = pd.to_datetime(numerical_df[['year', 'mon',
'day']].rename(columns={'year': 'year', 'mon': 'month', 'day': 'day'}))
numerical_df['Cases'] = pd.to_numeric(numerical_df['Cases'], errors='coerce')
numerical_df = numerical_df.dropna(subset=['Cases'])
numerical_df['Cases'] = numerical_df['Cases'].astype(int)
numerical_df = numerical_df.sort_values(by=['state_ut', 'district', 'Disease',
'date']).reset_index(drop=True)
numerical_df['next_week_cases'] = numerical_df.groupby(['state_ut', 'district',
'Disease'])['Cases'].shift(-1)
numerical_df = numerical_df.dropna(subset=['next_week_cases'])
numerical_df['next_week_cases'] = numerical_df['next_week_cases'].astype(int)

# Load original contextual data to create 'contextual_df'
contextual_data = []
with open(contextual_filepath, 'r') as f:
    for line in f:
        contextual_data.append(json.loads(line))
contextual_df = pd.DataFrame(contextual_data)
contextual_df['date'] = contextual_df['prompt'].apply(lambda x:
pd.to_datetime(re.search(r'Date: (\d{4}-\d{2}-\d{2})', x).group(1)))

# Create 'merged_df'
merged_df = pd.merge(contextual_df, numerical_df[['date', 'next_week_cases']], on='date',
how='inner')
print("DataFrames recreated successfully.")


# --- 2. Load Your Fine-Tuned Model ---
# Load the original Gemma model first
model_id = "google/gemma-2b-it"
bnb_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16)

tokenizer = AutoTokenizer.from_pretrained(model_id)
base_model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map={"":0}
)

# Merge your saved LoRA adapters into the base model
saved_model_path = "/kaggle/working/gemma-disease-forecast/final"
model = PeftModel.from_pretrained(base_model, saved_model_path)
model = model.merge_and_unload() # This makes inference faster
print("Fine-tuned model loaded successfully.")
```

```
# --- 3. Run the Final Evaluation ---
# This part is the same as before, but will now work because the DataFrames exist

test_prompts_df = contextual_df[contextual_df['date'].dt.year >= 2022]
test_prompts = test_prompts_df['prompt'].tolist()
true_values_df = merged_df[merged_df['date'].dt.year >= 2022]
y_true = true_values_df['next_week_cases'].tolist()

predictions = []
model.eval()

for prompt in test_prompts:
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=20,
pad_token_id=tokenizer.eos_token_id)
    result_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    match = re.search(r'expect around (\d+)', result_text)
    if match:
        predictions.append(int(match.group(1)))
    else:
        predictions.append(0)

min_len = min(len(predictions), len(y_true))
y_pred = predictions[:min_len]
y_true = y_true[:min_len]

fine_tuned_mae = mean_absolute_error(y_true, y_pred)

print("\n--- Final Comparison ---")
print(f"XGBoost Baseline MAE: 51.93")
print(f"Fine-Tuned Gemma MAE: {fine_tuned_mae:.2f}")# Define file paths
numerical_filepath = '/kaggle/input/final-dataset/Final_data.csv'
contextual_filepath = '/kaggle/input/disease-forecast-
corrected/disease_forecast_corrected.jsonl'

# Load numerical data to get the true future values
numerical_df = pd.read_csv(numerical_filepath)
numerical_df['date'] = pd.to_datetime(numerical_df[['year', 'mon',
'day']].rename(columns={'year': 'year', 'mon': 'month', 'day': 'day'}))
numerical_df['Cases'] = pd.to_numeric(numerical_df['Cases'], errors='coerce')
numerical_df = numerical_df.dropna(subset=['Cases'])
numerical_df['Cases'] = numerical_df['Cases'].astype(int)
numerical_df = numerical_df.sort_values(by=['state_ut', 'district', 'Disease',
'date']).reset_index(drop=True)
numerical_df['next_week_cases'] = numerical_df.groupby(['state_ut', 'district',
'Disease'])['Cases'].shift(-1)
numerical_df = numerical_df.dropna(subset=['next_week_cases'])
```

```
numerical_df['next_week_cases'] = numerical_df['next_week_cases'].astype(int)

# Load original contextual data to create 'contextual_df'
contextual_data = []
with open(contextual_filepath, 'r') as f:
    for line in f:
        contextual_data.append(json.loads(line))
contextual_df = pd.DataFrame(contextual_data)
contextual_df['date'] = contextual_df['prompt'].apply(lambda x:
pd.to_datetime(re.search(r'Date: (\d{4}-\d{2}-\d{2})', x).group(1)))

# Create 'merged_df'
merged_df = pd.merge(contextual_df, numerical_df[['date', 'next_week_cases']], on='date',
how='inner')
print("DataFrames recreated successfully.")


# --- 2. Load Your Fine-Tuned Model ---
# Load the original Gemma model first
model_id = "google/gemma-2b-it"
bnb_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16)

tokenizer = AutoTokenizer.from_pretrained(model_id)
base_model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map={"":0}
)

# Merge your saved LoRA adapters into the base model
saved_model_path = "/kaggle/working/gemma-disease-forecast/final"
model = PeftModel.from_pretrained(base_model, saved_model_path)
model = model.merge_and_unload() # This makes inference faster
print("Fine-tuned model loaded successfully.")


# --- 3. Run the Final Evaluation ---
# This part is the same as before, but will now work because the DataFrames exist

test_prompts_df = contextual_df[contextual_df['date'].dt.year >= 2022]
test_prompts = test_prompts_df['prompt'].tolist()
true_values_df = merged_df[merged_df['date'].dt.year >= 2022]
y_true = true_values_df['next_week_cases'].tolist()

predictions = []
model.eval()

for prompt in test_prompts:
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
```

```
    outputs = model.generate(**inputs, max_new_tokens=20,
pad_token_id=tokenizer.eos_token_id)
    result_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    match = re.search(r'expect around (\d+)', result_text)
    if match:
        predictions.append(int(match.group(1)))
    else:
        predictions.append(0)

min_len = min(len(predictions), len(y_true))
y_pred = predictions[:min_len]
y_true = y_true[:min_len]

fine_tuned_mae = mean_absolute_error(y_true, y_pred)

print("\n--- Final Comparison ---")
print(f"XGBoost Baseline MAE: 51.93")
print(f"Fine-Tuned Gemma MAE: {fine_tuned_mae:.2f}")
```

LLM fine-tuning on Disease specific Dataset:

```
# --- Load all 3 dengue datasets ---
features_train = pd.read_csv("/kaggle/input/dengue/dengue_features_train.csv")
labels_train = pd.read_csv("/kaggle/input/dengue/dengue_labels_train.csv")
features_test = pd.read_csv("/kaggle/input/dengue/dengue_features_test.csv")

# --- Merge features + labels for training ---
train_df = pd.merge(features_train, labels_train, on=["city", "year", "weekofyear"])

# --- Handle dates and missing values ---
train_df["week_start_date"] = pd.to_datetime(train_df["week_start_date"])
train_df.ffill(inplace=True)

print("Step 2 Complete: Datasets loaded and merged successfully!")
print(f"Training rows: {len(train_df)}, Test rows: {len(features_test)}")

print("\n--- Preview of the merged dengue training data ---")
display(train_df.head(3))

output_filename = "/kaggle/working/dengue_finetuning_data.jsonl"

with open(output_filename, "w") as f:
    for _, row in train_df.iterrows():
        city_name = "San Juan, Puerto Rico" if row["city"] == "sj" else "Iquitos, Peru"
        prompt = (
            f"Forecast the number of dengue cases for the week starting on
{row['week_start_date'].strftime('%Y-%m-%d')} "
```

```
            f"in {city_name}. Key environmental data includes: "
            f"Average Temperature: {row['station_avg_temp_c']:.2f}°C, "
            f"Total Precipitation: {row['station_precip_mm']:.2f} mm, "
            f"and satellite vegetation index (NDVI SE): {row['ndvi_se']:.4f}. "
            f"Based on this data, predict the total cases:"
        )
        completion = f"The predicted number of total cases is {int(row['total_cases'])}."
        record = {"prompt": prompt, "completion": completion}
        f.write(json.dumps(record) + "\n")

print(f"Step 3 Complete: Contextual dataset created → {output_filename}")

# Display one example
with open(output_filename, "r") as f:
    print("\nExample entry:\n", f.readline())

file_path = "/kaggle/working/dengue_finetuning_data.jsonl"

prompts, completions, texts = [], [], []

with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        try:
            rec = json.loads(line.strip())
            if isinstance(rec, dict):
                prompt = str(rec.get("prompt", "")).replace("\n", " ").strip()
                completion = str(rec.get("completion", "")).replace("\n", " ").strip()
                if prompt and completion:
                    prompts.append(prompt)
                    completions.append(completion)
                    texts.append(prompt + " " + completion)
        except Exception:
            continue

print(f"Loaded {len(texts)} valid prompt–completion pairs")

# Split 90% train / 10% validation
split_idx = int(len(texts) * 0.9)
train_texts = texts[:split_idx]
test_texts = texts[split_idx:]

print(f"Training samples: {len(train_texts)} | Validation samples: {len(test_texts)}")

# Display one sample
print("\nExample Prompt:")
print(prompts[0])
print("\nExample Completion:")
print(completions[0])

model_name = "microsoft/phi-2"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Add or assign a padding token
if tokenizer.pad_token is None:
    tokenizer.add_special_tokens({'pad_token': '[PAD]'})  # add new token if missing
tokenizer.pad_token = tokenizer.eos_token  # safer for causal models

print("Phi-2 tokenizer ready! pad_token:", tokenizer.pad_token)

def tokenize_texts(text_list, tokenizer):
    encodings = tokenizer(
        text_list,
        truncation=True,
        padding="max_length",
        max_length=512,
        return_tensors="pt"
    )
    return encodings

print("Tokenizing training data...")
train_encodings = tokenize_texts(train_texts, tokenizer)

print("Tokenizing validation data...")
test_encodings = tokenize_texts(test_texts, tokenizer)

print("Tokenization complete!")
print(f"Train input shape: {train_encodings['input_ids'].shape}")
print(f"Validation input shape: {test_encodings['input_ids'].shape}")


from transformers import (
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling,
    BitsAndBytesConfig
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from torch.utils.data import Dataset
import torch

# Wrap encodings in a Dataset
class DengueDataset(Dataset):
    def __init__(self, encodings):
        self.encodings = encodings
    def __len__(self):
        return self.encodings["input_ids"].shape[0]
    def __getitem__(self, idx):
        return {k: v[idx] for k, v in self.encodings.items()}
```

```
train_dataset = DengueDataset(train_encodings)
eval_dataset = DengueDataset(test_encodings)

# 4-bit quantization + CPU offload
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    llm_int8_enable_fp32_cpu_offload=True,
)

# Load Phi-2 in 4-bit
model_name = "microsoft/phi-2"
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)

# Prepare & add LoRA
model = prepare_model_for_kbit_training(model)
lora_config = LoraConfig(
    r=4,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

# Gradient checkpointing & disable cache
model.config.use_cache = False
model.gradient_checkpointing_enable()
for name, p in model.named_parameters():
    if "lora_" in name:
        p.requires_grad = True
model.print_trainable_parameters()

# Training configuration (extremely lightweight)
training_args = TrainingArguments(
    output_dir="/kaggle/working/phi2-dengue-finetuned",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=2,
    num_train_epochs=1,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=50,
    save_strategy="epoch",
    report_to=[]
```

```
)

data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    data_collator=data_collator,
)

# Train in ultra-low-memory mode
trainer.train()

# Step 7.1 — Save fine-tuned model and tokenizer
save_dir = "/kaggle/working/phi2-dengue-finetuned-final"

model.save_pretrained(save_dir)
tokenizer.save_pretrained(save_dir)

print(f"Fine-tuned model and tokenizer saved to {save_dir}")

# Paths
base_model_name = "microsoft/phi-2"
lora_dir = "/kaggle/working/phi2-dengue-finetuned-final"

# 4-bit config for GPU-efficient inference
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
)

# Load base + LoRA weights
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
model = PeftModel.from_pretrained(base_model, lora_dir)

# Tokenizer
tokenizer = AutoTokenizer.from_pretrained(lora_dir)
tokenizer.pad_token = tokenizer.eos_token

# Example dengue prediction prompt
prompt = (
    "Forecast the number of dengue cases for the week starting on 2009-04-20 "
    "in Iquitos, Peru. Key environmental data includes: "
```

```
    "Average Temperature: 26.4°C, Total Precipitation: 12.0 mm, "
    "and satellite vegetation index (NDVI SE): 0.2005. "
    "Based on this data, predict the total cases:"
)

inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
outputs = model.generate(
    **inputs,
    max_new_tokens=60,
    temperature=0.7,
    do_sample=False
)
print("Model Output:\n")
print(tokenizer.decode(outputs[0], skip_special_tokens=True))

# Load test dataset
test_df = pd.read_csv("/kaggle/input/dengue/dengue_features_test.csv")
test_df['week_start_date'] = pd.to_datetime(test_df['week_start_date'])
test_df.ffill(inplace=True)

# Make predictions
preds = []

for _, row in tqdm(test_df.iterrows(), total=len(test_df)):
    city_name = "San Juan, Puerto Rico" if row['city'] == 'sj' else "Iquitos, Peru"
    prompt = (
        f"Forecast the number of dengue cases for the week starting on
{row['week_start_date'].strftime('%Y-%m-%d')} "
        f"in {city_name}. Key environmental data includes: "
        f"Average Temperature: {row['station_avg_temp_c']:.2f}°C, "
        f"Total Precipitation: {row['station_precip_mm']:.2f} mm, "
        f"and satellite vegetation index (NDVI SE): {row['ndvi_se']:.4f}. "
        f"Based on this data, predict the total cases:"
    )

    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=40, temperature=0.7,
do_sample=False)
    text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Extract number from model output
    match = re.search(r"[-+]?\d*\.\d+|\d+", text)
    pred_value = float(match.group()) if match else 0.0
    preds.append(pred_value)

test_df["predicted_cases"] = preds

print("Finished generating predictions!")
test_df.head()
```

```
# Ensure model and tokenizer are loaded
model.eval()
tokenizer.pad_token = tokenizer.eos_token

# Prepare prompts and true labels from validation split
val_prompts = []
val_true = []

for entry in test_texts:  # 10% validation split
    match = re.search(r"The predicted number of total cases is (\d+)", entry)
    if match:
        val_true.append(float(match.group(1)))

for entry in test_texts:
    if "Based on this data" in entry:
        prompt = entry.split("Based on this data")[0] + (
            "Based on this data, predict the total cases. "
            "Do NOT include the year or other unrelated numbers. "
            "Only respond with the total number of dengue cases for that week:"
        )
    else:
        prompt = entry
    val_prompts.append(prompt)

print(f"Validation samples: {len(val_prompts)}")

# Generate predictions
preds = []
for prompt in tqdm(val_prompts, desc="Generating improved validation predictions"):
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True,
padding=True).to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=30, temperature=0.7,
do_sample=False)
    text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Extract the first numeric value from model output
    match = re.search(r"[-+]?\d*\.\d+|\d+", text)
    pred_value = float(match.group()) if match else 0.0

    # Optional: clip unrealistic outputs (e.g., >500 cases)
    if pred_value > 500:
        pred_value = 0.0

    preds.append(pred_value)

# Compute Mean Absolute Error (MAE)
mae = mean_absolute_error(val_true, preds)
print(f"\nFixed Validation MAE: {mae:.2f}")

# Combine and display results
```

```
results_df = pd.DataFrame({
    "True_Cases": val_true,
    "Predicted_Cases": preds
})
print("\nSample predictions after fix:")
print(results_df.head())

# Save clean results
results_df.to_csv("/kaggle/working/validation_results_phi2_fixed.csv", index=False)
print("Saved to: /kaggle/working/validation_results_phi2_fixed.csv")

model.eval()

preds = []
for prompt in tqdm(val_prompts, desc="Re-running validation with boosted sensitivity"):
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True,
padding=True).to("cuda")
    outputs = model.generate(
        **inputs,
        max_new_tokens=40,
        temperature=0.7,
        top_p=0.9,
        do_sample=False
    )
    text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Extract numeric value anywhere in the text
    numbers = re.findall(r"[-+]?\d*\.\d+|\d+", text)
    pred_value = float(numbers[-1]) if numbers else 0.0  # take last number (often the
prediction)

    # Clip extreme values
    if pred_value > 500:
        pred_value = 0.0

    preds.append(pred_value)

# Compute new MAE
mae = mean_absolute_error(val_true, preds)
print(f"\nValidation MAE (boosted): {mae:.2f}")

# results_df_boosted = pd.DataFrame({
#     "True_Cases": val_true,
#     "Predicted_Cases": preds
# })
# print("\n Sample predictions (boosted):")
# print(results_df_boosted.head())
```

8.3 FastAPI backend implementation (`main.py`)

```
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")

BASE = os.path.dirname(__file__)

# instantiate app
app = FastAPI(title="Disease & Case Prediction API", version="1.0")

# allow local dev CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
    allow_credentials=True,
)

# Model paths
MODEL_DIR = os.path.abspath(os.path.join(BASE, "..", "model-training"))
DISEASE_MODEL_PKL = os.path.join(MODEL_DIR, "disease_model.pkl")
MODEL_COLUMNS_PKL = os.path.join(MODEL_DIR, "model_columns.pkl")
UNIFIED_REG_PKL = os.path.join(MODEL_DIR, "unified_reg_model.pkl")
UNIFIED_REG_COLS_PKL = os.path.join(MODEL_DIR, "unified_reg_columns.pkl")
DISTRICT_STATS_CSV = os.path.join(MODEL_DIR, "disease_district_stats.csv")

# Try load models
def safe_load(path):
    try:
        return joblib.load(path)
    except Exception as e:
        logging.warning("Could not load %s : %s", path, e)
        return None

disease_model = safe_load(DISEASE_MODEL_PKL)
disease_model_columns = safe_load(MODEL_COLUMNS_PKL)
unified_reg_model = safe_load(UNIFIED_REG_PKL)
unified_reg_columns = safe_load(UNIFIED_REG_COLS_PKL)

# Load district lookup table
def safe_load_csv(path):
    try:
        return pd.read_csv(path)
    except Exception:
        return None

district_stats = safe_load_csv(DISTRICT_STATS_CSV)
```

```
def get_top_districts(disease, state, k=3):
    if district_stats is None:
        return []

    sub = district_stats[
        (district_stats["Disease"].str.lower() == str(disease).lower()) &
        (district_stats["state_ut"].str.lower() == str(state).lower())
    ]

    if sub.empty:
        return []

    return (
        sub.sort_values("Cases", ascending=False)
          .head(k)["district"]
          .tolist()
    )


# Pydantic schemas
class DiseaseInput(BaseModel):
    week_of_year: int
    state_ut: str
    temp_celsius: float
    preci: float

class CaseInput(DiseaseInput):
    Disease: str

# Helper to prepare input
def prepare_input(data_dict, model_columns):
    df = pd.DataFrame([data_dict])
    df_encoded = pd.get_dummies(df)

    if model_columns is None:
        return df_encoded

    df_final = df_encoded.reindex(columns=model_columns, fill_value=0)
    return df_final

# Endpoints
@app.post("/predict_disease")
def predict_disease(input_data: DiseaseInput):
    if disease_model is None or disease_model_columns is None:
        raise HTTPException(status_code=503, detail="Disease model not available.")

    try:
        input_df = prepare_input(input_data.dict(), disease_model_columns)
        pred = disease_model.predict(input_df)[0]
```

```
        try:
            proba = disease_model.predict_proba(input_df)
            confidence = float(np.max(proba))
        except:
            confidence = None

        top_risk = get_top_districts(str(pred), input_data.state_ut, k=3)

        return {
            "prediction": str(pred),
            "confidence": confidence,
            "top_districts": top_risk
        }

    except Exception as e:
        logging.exception("Error in predict_disease")
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/predict_cases")
def predict_cases(input_data: CaseInput):
    if unified_reg_model is None or unified_reg_columns is None:
        raise HTTPException(status_code=503, detail="Regression model not available.")

    try:
        input_df = prepare_input(input_data.dict(), unified_reg_columns)
        predicted_cases = float(unified_reg_model.predict(input_df)[0])

        return {
            "predicted_cases": round(predicted_cases, 2)
        }
    except Exception as e:
        logging.exception("Error in predict_cases")
        raise HTTPException(status_code=500, detail=str(e))


# Serve frontend
FRONTEND_DIR = os.path.abspath(os.path.join(BASE, "..", "frontend", "build"))
if os.path.isdir(FRONTEND_DIR):
    app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")

    @app.get("/", include_in_schema=False)
    def serve_index():
        index_file = os.path.join(FRONTEND_DIR, "index.html")
        if os.path.exists(index_file):
            return FileResponse(index_file)
        raise HTTPException(status_code=404, detail="Frontend index.html not found")

# Health check
@app.get("/health", include_in_schema=False)
```

```
def health():
    return {
        "ok": True,
        "disease_model_loaded": disease_model is not None,
        "reg_model_loaded": unified_reg_model is not None
    }

# Route listing
logging.info("Registered routes:")
for r in app.routes:
    try:
        logging.info(
            " - %s %s (include_in_schema=%s)",
            ",".join(r.methods) if hasattr(r, "methods") else "-",
            r.path,
            getattr(r, "include_in_schema", True)
        )
    except:
        pass
```

8.4 Dashboard (Frontend) Code Snippets

```
export default function App() {
    const [prediction, setPrediction] = useState(null);
    const [cases, setCases] = useState(null);
    const [form, setForm] = useState({});
    const [loading, setLoading] = useState(false);
    const [history, setHistory] = useState([]);

    const API_BASE = "http://127.0.0.1:8000";

    async function handlePredict(formData) {
        setForm(formData);
        setLoading(true);
        try {
            // 1) Predict disease
            const diseaseRes = await fetch(`${API_BASE}/predict_disease`, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify(formData),
            });
            const diseaseData = await diseaseRes.json();

            // 2) Predict cases for ±2 weeks
            const weekNum = Number(formData.week_no ?? formData.week ?? 0);
            const weekOffsets = [-2, -1, 0, 1, 2];
            const predictedCases = [];

            for (const offset of weekOffsets) {
                const weekForm = {
```

```
            ...formData,
            Disease: diseaseData.prediction,
            week_no: weekNum + offset,
          };
          const res = await fetch(`${API_BASE}/predict_cases`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(weekForm),
          });
          const data = await res.json();
          predictedCases.push({
            week: weekNum + offset,
            predicted_cases: data.predicted_cases,
          });
        }

        // 3) Store results
        setPrediction(diseaseData);
        setCases(predictedCases);

        setHistory((prev) => [
          ...prev,
          {
            ...formData,
            disease: diseaseData.prediction,
            confidence: diseaseData.confidence,
            predicted_series: predictedCases,
          },
        ]);
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    }

    return (
      <div className="flex h-screen bg-gray-50">
        <Sidebar />

        <div className="flex flex-col flex-1 overflow-y-auto">
          <Header />

          <main className="p-6 space-y-6">
            <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">

              <div className="lg:col-span-1">
                <PredictionForm
                  onSubmit={handlePredict}
                  loading={loading}
```

```
            />
          </div>

          <div className="lg:col-span-2 space-y-6">

            <DashboardCards
              prediction={prediction}
              cases={cases}
            />

            <ChartsPanel
              data={history}
            />

            <IndiaMap
              geojson={indiaGeo}
              targetState={form.state_ut}
              atRiskDistricts={prediction?.top_districts ?? []}
            />

            {/* At-Risk Districts Display */}
            {prediction?.top_districts && prediction.top_districts.length > 0 ? (
              <div className="bg-white shadow rounded-lg p-4">
                <h3 className="text-lg font-semibold mb-2">
                  ⚠ At-Risk Districts
                </h3>
                <ul className="list-disc ml-5 text-sm space-y-1">
                  {prediction.top_districts.map((district, i) => (
                    <li key={i} className="text-red-600 font-medium">
                      {district}
                    </li>
                  ))}
                </ul>
              </div>
            ) : (
              <div className="bg-white shadow rounded-lg p-4">
                <h3 className="text-lg font-semibold mb-2">
                  ⚠ At-Risk Districts
                </h3>
                <p className="text-gray-500 text-sm">
                  No data available
                </p>
              </div>
            )}

          </div>
        </div>
      </main>
    </div>
  </div>
```

```
    );
}
```

8.5 Final Classifying Code:

```
# --- 1. Load the Dataset ---
# Always good to start with a try-except block to handle file errors.
try:
    df = pd.read_csv('/content/Final_data.csv')
    print("File loaded successfully!")
except FileNotFoundError:
    print("Error: Final_data.csv not found. Please make sure the file is in the same directory.")
    exit()

# --- 2. Clean Individual Columns ---

# Clean 'week_of_outbreak': Extract just the number from strings like "1st week", "52nd
week", etc.
# We use a regular expression to find the first sequence of digits.
df['week_of_year'] = df['week_of_outbreak'].str.extract('(\d+)').astype(int)

# Clean 'Temp': Convert the temperature from Kelvin to Celsius.
# The formula is Celsius = Kelvin - 273.15
df['temp_celsius'] = df['Temp'] - 273.15

# Clean 'Deaths': The preview showed missing values. A simple strategy is to
# fill these missing values with 0, assuming 'no data' means 'no reported deaths'.
df['Deaths'] = df['Deaths'].fillna(0)

# --- 3. Select Columns for the Model ---
# We will select only the columns we need for our classification task.
# This makes our dataset clean and focused.
model_data = df[[
    'Disease',
    'week_of_year',
    'district',
    'state_ut',
    'temp_celsius',
    'preci' # Precipitation
]]

# --- 4. Final Touches ---
# Drop any rows that might still have missing values in our selected columns.
model_data = model_data.dropna()

# Display the first few rows of the cleaned data and its info to confirm changes.
print("\n--- Cleaned Data Information ---")
model_data.info()

print("\n--- First 5 Rows of Cleaned Data ---")
```

```
print(model_data.head())

# Optional: Save the cleaned data to a new CSV file for the next step.
model_data.to_csv('cleaned_disease_data.csv', index=False)
print("\nCleaned data saved to 'cleaned_disease_data.csv'")

"""training the model."""

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- 1. Load the Cleaned Dataset ---
try:
    df = pd.read_csv('/content/cleaned_disease_data.csv')
    print("Cleaned data loaded successfully!")
except FileNotFoundError:
    print("Error: 'cleaned_disease_data.csv' not found. Please make sure you ran the Step 1
script.")
    exit()

# --- 2. Define Features (X) and Target (y) ---
# The target is what we want to predict.
target = 'Disease'

# The features are the inputs the model will use to make the prediction.
# We'll use all columns from our cleaned data except the target itself.
features = ['week_of_year', 'district', 'state_ut', 'temp_celsius', 'preci']

X = df[features]
y = df[target]

# --- 3. Handle Categorical Data using One-Hot Encoding ---
# Machine learning models require all input to be numeric.
# pd.get_dummies() converts columns like 'district' and 'state_ut' into a numeric format.
X_encoded = pd.get_dummies(X, columns=['district', 'state_ut'], drop_first=True)

# --- 4. Split Data for Training and Testing ---
# We use 80% of the data to train the model and 20% to test its performance.
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2,
random_state=42)

print(f"\nTraining data has {X_train.shape[0]} samples.")
print(f"Testing data has {X_test.shape[0]} samples.")

# --- 5. Train the Random Forest Model ---
# We initialize the classifier and then fit it to our training data.
print("\nTraining the model...")
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1) #
n_jobs=-1 uses all available CPU cores
model.fit(X_train, y_train)
print("Model training complete!")

# --- 6. Evaluate the Model's Performance ---
print("\n--- Model Evaluation ---")
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, predictions))

# Add this to the end of your train_model.py script
import joblib

# --- 7. Save the Model and Columns for Future Use ---
joblib.dump(model, 'disease_model.pkl')
joblib.dump(X_encoded.columns, 'model_columns.pkl')

print("\nModel and column names have been saved!")

import pandas as pd
import joblib

# --- 1. Load the Saved Model and Columns ---
try:
    model = joblib.load('disease_model.pkl')
    model_columns = joblib.load('model_columns.pkl')
    print("Model loaded successfully!")
except FileNotFoundError:
    print("Error: Model files not found. Please run the updated training script first.")
    exit()

# --- 2. Create a DataFrame for the New Prediction ---
# This is where we input the data for "next week in Amaravati".
# Note: The district for Amaravati is 'Guntur' in our original dataset.
new_data = {
    'week_of_year': 40,  # The first week of October is the 40th week
    'district': 'Guntur',
    'state_ut': 'Andhra Pradesh',
    'temp_celsius': 28.5, # Our estimated temperature
    'preci': 50.0 # Our estimated precipitation
}

# Convert the dictionary to a pandas DataFrame
input_df = pd.DataFrame([new_data])

# --- 3. Prepare the Input Data ---
```

```
# We must one-hot encode the new data exactly like the training data.
input_encoded = pd.get_dummies(input_df)

# The key step: ensure the new data has the exact same columns as the training data.
# The .reindex() method will add any missing columns and fill them with 0.
final_input = input_encoded.reindex(columns=model_columns, fill_value=0)

# --- 4. Make the Prediction ---
prediction = model.predict(final_input)
probability = model.predict_proba(final_input)

print("\n--- Prediction for Next Week ---")
print(f"The most likely disease in Amaravati (Guntur district) next week is:
**{prediction[0]}**")
print(f"Confidence: {np.max(probability) * 100:.2f}%")

import pandas as pd

# --- 1. Load the Original Dataset ---
try:
    df = pd.read_csv('Final_data.csv')
    print("File loaded successfully!")
except FileNotFoundError:
    print("Error: Final_data.csv not found.")
    exit()

# --- 2. Robustly Clean 'Cases' Column ---
df['Cases'] = pd.to_numeric(df['Cases'], errors='coerce')
df.dropna(subset=['Cases'], inplace=True)
df['Cases'] = df['Cases'].astype(int)

# --- 3. EXCLUDE PANDEMIC YEARS (2020-2022) ---
# This is the corrected and most important step.
initial_rows = len(df)
pandemic_years = [2020, 2021, 2022]
df = df[~df['year'].isin(pandemic_years)].copy()
print(f"Removed {initial_rows - len(df)} records from the pandemic years (2020-2022).")


# --- 4. Perform Other Cleaning Steps ---
df['week_of_year'] = df['week_of_outbreak'].str.extract('(\d+)').astype(int)
df['temp_celsius'] = df['Temp'] - 273.15
df['Deaths'] = df['Deaths'].fillna(0)


# --- 5. Select Final Columns for Modeling ---
final_columns = [
    'Disease',
    'Cases',
    'week_of_year',
```

```
    'district',
    'state_ut',
    'temp_celsius',
    'preci',
    'Latitude',
    'Longitude'
]
model_data = df[final_columns].dropna()


# --- 6. Save the Final Cleaned Data ---
model_data.to_csv('cleaned_disease_data.csv', index=False)
print("\n--- Final Cleaned Data (Excluding Pandemic Period) ---")
print(f"Created a final dataset with {len(model_data)} records.")
print("\nFinal cleaned data saved to 'cleaned_disease_data.csv'")

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# --- 1. Load the Full Cleaned Dataset ---
try:
    df = pd.read_csv('cleaned_disease_data.csv')
    print("Full cleaned dataset loaded successfully!")
except FileNotFoundError:
    print("Error: 'cleaned_disease_data.csv' not found. Please make sure the Step 1 cleaning
script was run.")
    exit()

# --- 2. Define Features (X) and Target (y) ---
# The target is still 'Cases'.
target = 'Cases'

# The features now INCLUDE the 'Disease' column.
features = ['week_of_year', 'district', 'state_ut', 'temp_celsius', 'preci', 'Disease']

X = df[features]
y = df[target]

# --- 3. One-Hot Encode ALL Categorical Features ---
# This time, we include 'Disease' in the list of columns to encode.
X_encoded = pd.get_dummies(X, columns=['district', 'state_ut', 'Disease'], drop_first=True)

# --- 4. Split Data for Training and Testing ---
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2,
random_state=42)

# --- 5. Train the Unified Random Forest Regressor ---
print("\nTraining the unified regression model...")
```

```
unified_reg_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-
1)
unified_reg_model.fit(X_train, y_train)
print("Model training complete!")

# --- 6. Evaluate the Unified Model's Performance ---
print("\n--- Unified Regression Model Evaluation ---")
predictions = unified_reg_model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"R-squared (R²): {r2:.2f}")
```

CHAPTER 9

REFERENCES

Research & Disease Forecasting

1. World Health Organization, "Climate Change and Health," WHO, 2022.
2. C. J. Murray et al., "Global, regional, and national incidence and mortality for infectious diseases," *The Lancet*, 2020.
3. L. Lowe et al., "Dengue forecasting using climate data," *Nature Scientific Reports*, 2018.
4. Siraj et al., "Temperature explains a large fraction of the variation in dengue incidence," *PNAS*, 2017.

Machine Learning

5. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
6. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM KDD*, 2016.
7. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
8. K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder (GRU)," arXiv:1406.1078, 2014.

Large Language Models

9. Gemma Team, "Gemma: Open small language models," Google DeepMind, 2024.
10. E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021.
11. Touvron et al., "LLaMA: Open and efficient foundation language models," Meta AI, 2023.

Datasets

12. UCI Machine Learning Repository, Dengue Fever Dataset (San Juan & Iquitos), 2018.
13. Government of India, Integrated Disease Surveillance Programme (IDSP), epidemiological data, 2023.
14. Indian Meteorological Department (IMD), rainfall & temperature datasets, 2023.
15. EpiClim: Weekly District-Wise all-India multi-epidemics Climate-Health Dataset for accelerated GeoHealth research

Frameworks & Tools

16. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, 2011.
17. The FastAPI Authors, "FastAPI Documentation," fastapi.tiangolo.com.
18. React Documentation, Meta Platforms, 2024.
19. Kaggle Inc., "Kaggle Notebook GPU Resources," kaggle.com.
20. Python Software Foundation, "Python Language Reference," python.org

BIODATA



Name : Chinnola Akash Rao

Mobile Number : 9392179343

E-mail : chinolaakash@gmail.com

Permanaent Address : 36-A-196, papireddy Nagar, jagathgiri gutta, Quthbullapur, Hyderabad, Telangana 500037



Name : Aditya Nair

Mobile Number : 8452937731

E-mail : nairaditya88@gmail.com

Permanaent Address : Panchavati society,Marol Maroshi Road,Marol,Andheri East, Mumbai 400059



Name : Nandana S Krishna

Mobile Number : 7306691334

E-mail : nandanaskrishna030704@gmail.com

Permanaent Address : Vadakkedathu (H), Kumaramangalam P.O, Thodupuzha, Idukki, Kerala



Name : Edhara Jnaneswar Chowdary

Mobile Number : 8522990114

E-mail : jnaneswaredhara@gmail.com

Permanaent Address : 1-90/A, near masjid street,Kalavalapalli Chagallu Mandal, West Godavari